VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
FACULTY OF COMPUTER SCIENCE AND ENGINEERING



Machine Learning (CO3117)

Assignment Report

Credit Risk Prediction with Traditional Machine Learning, Deep Learning and Bayesian Networks

---

Advisor(s): Dr. Le Thanh Sach
Student(s): Duong Nguyen Tan Dat 2352222

HO CHI MINH CITY, OCTOBER 2026

Member list & workload

No.	Full name	Student ID	Contribution
1	Duong Nguyen Tan Dat	2352222	100%

The group has a single member, who carried out every task listed below.

Work item	Description	Done
Dataset selection and EDA	Data quality audit, distributions, correlations, missing-value analysis	100%
Traditional pipeline	Configurable preprocessing, six ablation suites, tuning of eight classifiers	100%
Deep learning pipeline	Entity-embedding MLP (seven variants), TabNet, learned features	100%
Bayesian network (extension)	From-scratch DAG, learning, exact and approximate inference	100%
Engineering	Python package, unit tests, Colab notebook, GitHub repository	100%
Report	This document	100%

Abstract

This report predicts whether a borrower will default on a loan, using the public Credit Risk Dataset of 32 581 loans. Three approaches are built and compared under one evaluation protocol designed for an imbalanced problem in which a missed defaulter is costlier than a rejected applicant: models are ranked by PR-AUC and their decision thresholds are tuned for the F2 score. First, a fully configurable traditional pipeline is evaluated through controlled ablations of imputation, outlier handling, encoding, scaling, dimensionality reduction and class-imbalance strategies, followed by the tuning of eight classifiers. Second, a deep learning pipeline trains a multilayer perceptron with entity embeddings and TabNet. Third, as the talented-engineering extension, a Bayesian network library is implemented from scratch, with Naive Bayes, TAN, BIC hill-climbing and expert structures, exact inference by variable elimination and approximate inference by rejection sampling, likelihood weighting and Gibbs sampling. Histogram-based gradient boosting performs best, with a test PR-AUC of 0.9045; at its F2-tuned threshold it detects 85.1% of the defaulters with a precision of 69.0%, and neither a wider search nor stacking improves on it. The entity-embedding network outperforms every linear model but not the tree ensembles. The Bayesian networks are less accurate but fully interpretable: they answer diagnostic queries exactly, and comparing their answers with empirical frequencies reveals which independence assumptions the data contradict. All experiments run end to end in a Google Colab notebook, and the code, features and report are available on GitHub.

Contents

Member list & workload **3**

1 Introduction **10**
1.1 Problem statement . . . 10
1.2 Objectives . . . 10
1.3 Contributions . . . 11
1.4 Deliverables and reproducibility . . . 11
1.5 Organization of the report . . . 11

2 Dataset and exploratory data analysis **12**
2.1 The Credit Risk Dataset . . . 12
2.2 Data quality . . . 13
2.3 Target distribution . . . 14
2.4 Numeric features . . . 15
2.5 Categorical features . . . 16
2.6 Correlations and feature relevance . . . 16
2.7 Missing values . . . 18
2.8 Implications for the pipeline . . . 19

3 Methodology **20**
3.1 Evaluation protocol . . . 21
3.2 Traditional machine learning pipeline . . . 22
3.2.1 Missing-value imputation . . . 23
3.2.2 Feature engineering and outliers . . . 23
3.2.3 Encoding and scaling . . . 24
3.2.4 Feature selection and dimensionality reduction . . . 24
3.2.5 Class imbalance . . . 24
3.2.6 Classifiers . . . 24
3.3 Deep learning pipeline . . . 25
3.4 Bayesian network (extension for the talented-engineering program) . . . 26
3.4.1 Discretization and missing values . . . 27
3.4.2 Parameter learning . . . 27
3.4.3 Structures . . . 27
3.4.4 Exact inference: variable elimination . . . 28
3.4.5 Approximate inference . . . 28

3.4.6 Classification with a Bayesian network 29

4 Experiments and results **29**
4.1 Setup 29
4.2 Handling impossible values 30
4.3 Ablation studies of the traditional pipeline 30
4.3.1 Imputation 31
4.3.2 Scaling 31
4.3.3 Outliers and feature engineering 33
4.3.4 Encoding of the loan grade 33
4.3.5 Dimensionality reduction and feature selection 34
4.3.6 Class imbalance 36
4.4 Tuned comparison of the traditional models 36
4.4.1 Is there headroom left? 39
4.4.2 Feature importance and error analysis 39
4.5 Deep learning pipeline 40
4.5.1 MLP variants 40
4.5.2 TabNet and comparison with the traditional pipeline 41
4.6 Bayesian networks 43
4.6.1 Discretization and smoothing 43
4.6.2 Structures and classification 44
4.6.3 Probabilistic reasoning 46
4.6.4 Exact versus approximate inference 47
4.7 Overall comparison 48

5 Discussion **49**
5.1 Comparison of the three approaches 49
5.2 What mattered most 50
5.3 Choice of metric 51
5.4 Limitations and threats to validity 51

6 Conclusion and future work **52**

A Project structure and usage **56**

B Unit tests **56**

C Saved feature files **57**

List of Figures

2.1 Box plots of the raw numeric features. Note the impossible ages and employment lengths, and the heavy right tails of income and amounts. 14
2.2 Distribution of the target `loan_status`. 15
2.3 Class-conditional distributions of the numeric features (after removing impossible values; densities normalized per class). 15
2.4 Categorical features: counts (top) and default rate per category (bottom; the dashed line is the overall rate). 16
2.5 Spearman correlations between numeric features, the ordinal grade, the past-default flag and the target. 17
2.6 Relevance of individual features and an example interaction. 18
2.7 Left: default rate when a value is missing or observed. Right: missing share of `loan_int_rate` per grade. 19
2.8 Interest rate by loan grade: the grade almost determines the rate. 19
3.1 Overview of the project: one cleaned and split dataset feeds three model families that share the same evaluation protocol. 21
4.1 Effect of the scaler on PR-AUC (5-fold CV; error bars show one standard deviation). 32
4.2 PCA and feature selection lower PR-AUC for every model (5-fold CV). . . 35
4.3 ROC (left) and precision-recall (right) curves of the tuned traditional models on the test set. 38
4.4 Confusion matrices on the test set at each model's F2-tuned threshold. . . 38
4.5 Permutation importance of the best traditional model on the test set (drop in PR-AUC). 40
4.6 Training loss and validation PR-AUC of the reference MLP (seed 42). . . . 41
4.7 Best traditional model, logistic regression, MLP and TabNet (seed 42) on the test set. 42
4.8 TabNet's attention-based feature importance (seed 42). 42
4.9 The four Bayesian-network structures (DEFAULT is the class node). 45
4.10 Mean absolute error of approximate inference against variable elimination (log-log scale). 48
4.11 All models ranked by test PR-AUC; the error bars of the deep models are the standard deviation over five seeds. 49

List of Tables

2.1 Columns of the Credit Risk Dataset. 12
2.2 Columns of the raw dataset. 13
2.3 Descriptive statistics of the numeric features (raw data). 14
2.4 From EDA findings to pipeline options. 20
3.1 Classifiers available in the traditional pipeline. 25
3.2 Bayesian-network modules written for this project. 26
4.1 Handling the 7 impossible training values: 5-fold CV PR-AUC on identical validation rows (mean fold standard deviation 0.0052). 30
4.2 Imputation strategies: 5-fold CV on the training set (F2, recall and precision at threshold 0.5). 31
4.3 Scaling methods: 5-fold CV on the training set (F2, recall and precision at threshold 0.5). 32
4.4 Outlier handling and feature engineering: 5-fold CV on the training set (F2, recall and precision at threshold 0.5). 33
4.5 Encoding of the loan grade: 5-fold CV on the training set (F2, recall and precision at threshold 0.5). 34
4.6 PCA and univariate feature selection: 5-fold CV on the training set (F2, recall and precision at threshold 0.5). 35
4.7 Class-imbalance strategies: 5-fold CV on the training set (F2, recall and precision at threshold 0.5). 36
4.8 Grid-search results (5-fold CV, refit on PR-AUC; F2 at threshold 0.5). . . 37
4.9 Test-set results of the tuned traditional models; F2, recall, precision and accuracy at the F2-tuned threshold. 37
4.10 Headroom check: stronger optimization of the best model family (5-fold CV on the training set). 39
4.11 MLP variants: validation PR-AUC (used for selection) and test metrics, mean ± sd over 5 seeds (epochs and training time are per-seed means). . . 41
4.12 Deep learning models on the test set, mean ± sd over 5 seeds (threshold tuned for F2 on a validation split; training time per seed). 42
4.13 Bayesian networks: validation PR-AUC by number of bins and Dirichlet pseudo-count α. 43
4.14 Bayesian-network structures on the test set, each with the bins and α selected on validation data. 45



4.15 Exact answers of two networks (variable elimination) against the empirical frequency in the training data; support is the number of training rows matching the evidence. 46
4.16 Mean absolute error of approximate $P(\text{default} \mid \boldsymbol{e})$ against variable elimination (60 random test applicants per evidence size). 47
4.17 Acceptance rate of rejection sampling. 48
4.18 All model families on the test set, ranked by PR-AUC (F2, recall, precision and accuracy at the tuned threshold); deep models are mean ± sd over 5 seeds. 48
5.1 Strengths and limitations of the three model families on this problem. . . . 50
C.1 Extracted feature sets (`X_train`, `X_test`, `y_train`, `y_test` in each). 57

List of Algorithms

3.1 Variable elimination for $P(Q \mid \boldsymbol{e})$ 28
3.2 Likelihood weighting for $P(Q \mid \boldsymbol{e})$ 29

1 Introduction

1.1 Problem statement

Before granting a loan, a lender must estimate how likely the applicant is to default, that is, to stop repaying. Approving a future defaulter loses the principal, while rejecting a reliable applicant only loses the interest margin, so the two kinds of error have very different costs. This report studies the problem as binary classification on the public Credit Risk Dataset[16]. Each of its 32 581 records describes one applicant (age, income, employment length, home ownership, credit history) and one loan (purpose, grade, amount, interest rate, loan-to-income ratio). The target `loan_status` equals 1 when the loan defaulted.

The dataset meets the constraints of the assignment for tabular data. It contains missing values (in `person_emp_length` and `loan_int_rate`), which calls for imputation. It contains four categorical variables, which call for encoding. It is large enough for reliable cross-validation and a held-out test set of 6484 loans.

1.2 Objectives

Following the assignment specification, the project pursues four objectives.

1. **Traditional pipeline (mandatory).** Build a complete pipeline with exploratory data analysis, preprocessing, feature extraction and selection, model training and evaluation, where every step is configurable. Then measure the effect of each choice with controlled experiments.

2. **Model comparison.** Tune and compare eight classical classifiers under one evaluation protocol designed for an imbalanced, cost-asymmetric problem.

3. **Deep learning pipeline (bonus).** Train neural networks that learn their own representation of the tabular data (an entity-embedding MLP and TabNet), save the learned features, and compare them with the traditional pipeline.

4. **Bayesian network (talented-engineering extension).** Implement a Bayesian network from scratch, with structure and parameter learning, exact inference (variable elimination) and approximate inference (rejection sampling, likelihood weighting, Gibbs sampling). Then apply it to the same prediction problem and to probabilistic reasoning about the causes of default.



1.3 Contributions

- A reusable Python package (`modules/creditrisk`) that builds any pipeline variant from a declarative configuration. It is driven by a Google Colab notebook that runs end to end with *Run all*.
- Six ablation studies on imputation, scaling, outliers and feature engineering, encoding, dimensionality reduction, and class imbalance, followed by grid-searched models with F2-tuned decision thresholds.
- A deep learning comparison with seven MLP variants and TabNet, and an analysis of the learned embeddings.
- A from-scratch Bayesian-network library, verified by unit tests against brute-force enumeration and scikit-learn's `CategoricalNB`. It is used to compare four network structures and three sampling algorithms.
- Saved feature sets (`.npy` and `.h5`) for every pipeline, and a public GitHub repository.

1.4 Deliverables and reproducibility

The submission follows the required layout: `notebooks/`, `modules/`, `reports/` and `features/`.

- Source code: https://github.com/datduongabc/credit-risk-classification
- Colab notebook: https://colab.research.google.com/github/datduongabc/credit-risk-classification/blob/main/notebooks/credit_risk_pipeline.ipynb

The notebook clones the repository, installs the missing libraries and downloads the dataset from a public link, without mounting any personal storage. Every number in this report is read from files that the notebook generates (`reports/generated/`). All randomness is seeded, so a re-run reproduces the tables; the deep models, whose results vary slightly across hardware and PyTorch builds, are reported as the mean and standard deviation over five seeds, so a run on another machine lands within the reported spread.

1.5 Organization of the report

Section 2 analyses the data and derives the preprocessing decisions. Section 3 presents the evaluation protocol and the three modelling approaches. Section 4 reports the ex-

periments. Section 5 discusses strengths, limitations and threats to validity. Section 6 concludes. The appendix documents the code structure, the tests and the saved features.

2 Dataset and exploratory data analysis

2.1 The Credit Risk Dataset

The dataset[16] has 32 581 rows and 12 columns: seven numeric features, four categorical features and the binary target (table 2.2). The dataset is released under CC0 (public domain). The notebook downloads it from a public mirror in the project's GitHub releases, falls back to Kaggle, and verifies its SHA-256 checksum, so every run uses exactly the same file.

Column	Meaning
`person_age`	Age of the applicant (years)
`person_income`	Annual income (USD)
`person_home_ownership`	RENT, MORTGAGE, OWN or OTHER
`person_emp_length`	Employment length (years)
`loan_intent`	Purpose: EDUCATION, MEDICAL, VENTURE, PERSONAL, DEBTCONSOLIDATION, HOMEIMPROVEMENT
`loan_grade`	Lender's risk grade from A (best) to G (worst)
`loan_amnt`	Requested amount (USD)
`loan_int_rate`	Interest rate (%)
`loan_percent_income`	Loan amount divided by annual income
`cb_person_default_on_file`	Whether the credit bureau records a past default (Y/N)
`cb_person_cred_hist_length`	Length of the credit history (years)
`loan_status`	**Target**: 1 = default, 0 = non-default

Table 2.1: Columns of the Credit Risk Dataset.

column	type	missing	missing (%)	unique
person_age	numeric	0	0.00	58
person_income	numeric	0	0.00	4295
person_home_ownership	categorical	0	0.00	4
person_emp_length	numeric	895	2.75	36
loan_intent	categorical	0	0.00	6
loan_grade	categorical	0	0.00	7
loan_amnt	numeric	0	0.00	753
loan_int_rate	numeric	3116	9.56	348
loan_status	categorical	0	0.00	2
loan_percent_income	numeric	0	0.00	77
cb_person_default_on_file	categorical	0	0.00	2
cb_person_cred_hist_length	numeric	0	0.00	29

Table 2.2: Columns of the raw dataset.

2.2 Data quality

Duplicates. 165 rows are exact duplicates of other rows. They are removed, because duplicates inflate the apparent sample size and can land on both sides of the train/test split.

Impossible values. Five applicants are older than 100 (two of them are 144), and two report 123 years of employment while being 21 and 22 years old (fig. 2.1). These are data-entry errors, not genuine extremes. We turn them into missing values, using the rules age > 100 and employment length $>$ age $- 14$, and let the imputer fill them. The rules are deterministic and row-local, so applying them before the split cannot leak information.

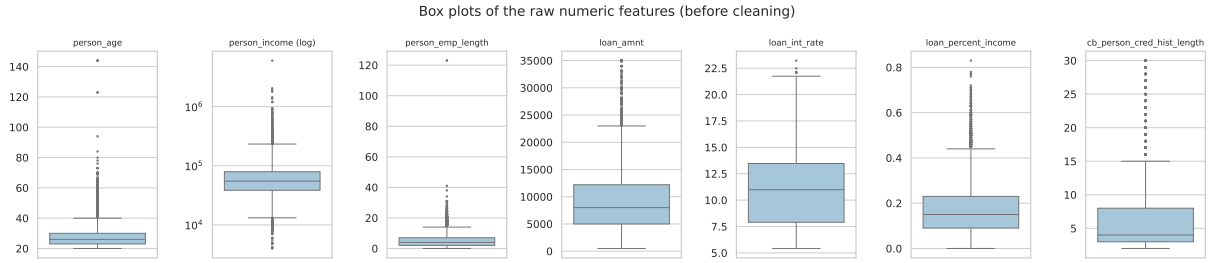


Figure 2.1: Box plots of the raw numeric features. Note the impossible ages and employment lengths, and the heavy right tails of income and amounts.

Skewness and genuine outliers. Income is extremely right-skewed (skewness 32.9, median \$55,000, maximum \$6,000,000). The loan amount, loan-to-income ratio and credit-history length also have long right tails. Unlike the impossible values, these are plausible and are kept. Whether to winsorize them or log-transform them is treated as a pipeline option and decided experimentally (table 4.4).

feature	mean	std	min	1%	50%	99%	max	skew
person_age	27.73	6.35	20.00	21.00	26.00	50.00	144.00	2.58
person_income	66074.85	61983.12	4000.00	14400.00	55000.00	225200.00	6000000.00	32.87
person_emp_length	4.79	4.14	0.00	0.00	4.00	18.00	123.00	2.61
loan_amnt	9589.37	6322.09	500.00	1000.00	8000.00	29800.00	35000.00	1.19
loan_int_rate	11.01	3.24	5.42	5.42	10.99	18.62	23.22	0.21
loan_percent_income	0.17	0.11	0.00	0.02	0.15	0.50	0.83	1.06
cb_person_cred_hist_length	5.80	4.06	2.00	2.00	4.00	17.00	30.00	1.66

Table 2.3: Descriptive statistics of the numeric features (raw data).

2.3 Target distribution

Only 21.8% of the loans defaulted (fig. 2.2). This moderate imbalance drives three design decisions:

- the split and every cross-validation fold are stratified;
- accuracy is not used for model selection, PR-AUC and F2 are used instead (section 3.1);
- class weights, SMOTE and under-sampling are compared as remedies.

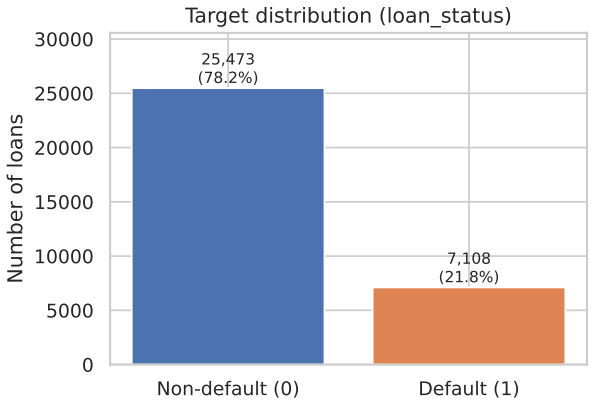


Figure 2.2: Distribution of the target `loan_status`.

2.4 Numeric features

Figure 2.3 compares the class-conditional distributions after cleaning. Three features separate the classes clearly.

- `loan_percent_income`: defaulters typically borrow more than 30% of their income, while most non-defaulters stay below 20%.
- `loan_int_rate`: defaulters concentrate at rates above 14%.
- `person_income`: defaulters have lower incomes (on the log scale).

Age, employment length and credit-history length have nearly identical distributions in both classes. Most applicants are young (median age 26; 76% are 30 or younger).

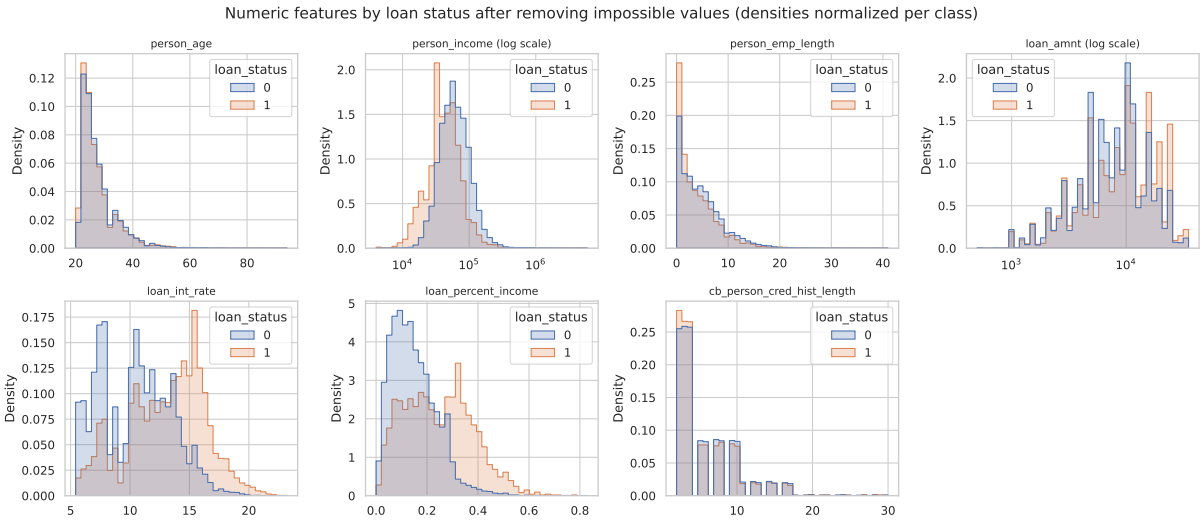


Figure 2.3: Class-conditional distributions of the numeric features (after removing impossible values; densities normalized per class).

2.5 Categorical features

Figure 2.4 shows the frequency of each category and its default rate.

- **Loan grade** is the strongest categorical signal. The default rate rises monotonically from 10.0% (A) and 16.3% (B) to 59.0% (D) and 98.4% (G). The jump between C and D is the largest. The order supports an ordinal encoding.
- **Home ownership**: renters default four times as often as owners (31.6% versus 7.5%). Mortgage holders sit in between (12.6%). The OTHER category is rare (107 rows).
- **Past default on file**: 37.8% of applicants with a recorded past default default again, against 18.4% of the others.
- **Loan intent** has a weaker effect, from 14.8% (VENTURE) to 28.6% (DEBTCONSOLIDATION).

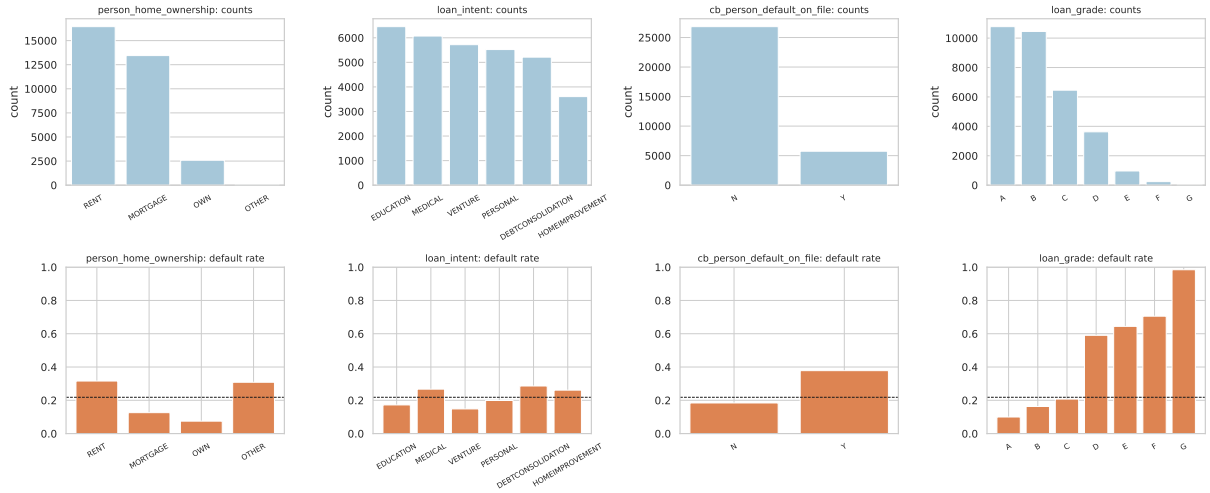


Figure 2.4: Categorical features: counts (top) and default rate per category (bottom; the dashed line is the overall rate).

2.6 Correlations and feature relevance

The Spearman correlation matrix (fig. 2.5) reveals three strongly related pairs:

- the interest rate and the grade ($\rho = 0.95$): the lender prices loans by grade;
- age and credit-history length ($\rho = 0.81$);
- the loan amount and the loan-to-income ratio ($\rho = 0.65$).

These redundancies matter for linear models (collinearity), and they explain why a single grade-related feature carries most of the signal. The features most correlated with the target are the grade (0.33), the interest rate (0.32), the loan-to-income ratio (0.32) and

the income (-0.27).

Mutual information (fig. 2.6a) also captures non-monotone dependence. It ranks income first, followed by the loan-to-income ratio, the grade and the interest rate. Age and credit-history length carry almost no information on their own. The scatter plot in fig. 2.6b shows that the classes overlap strongly in any single feature, but that defaults concentrate where low income meets a high loan-to-income ratio. This is an interaction that tree ensembles can capture directly.

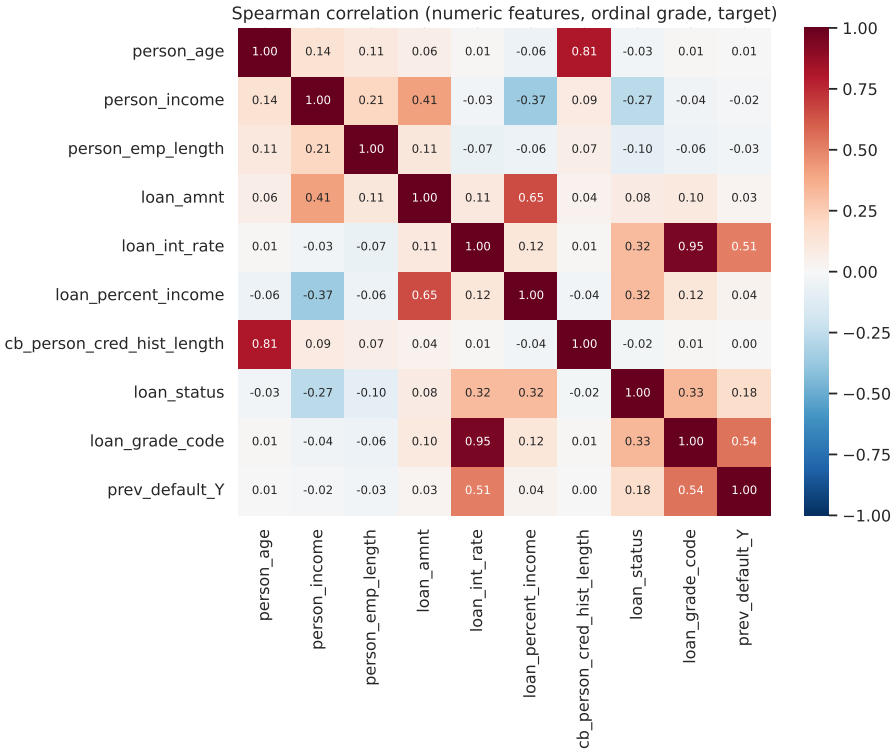
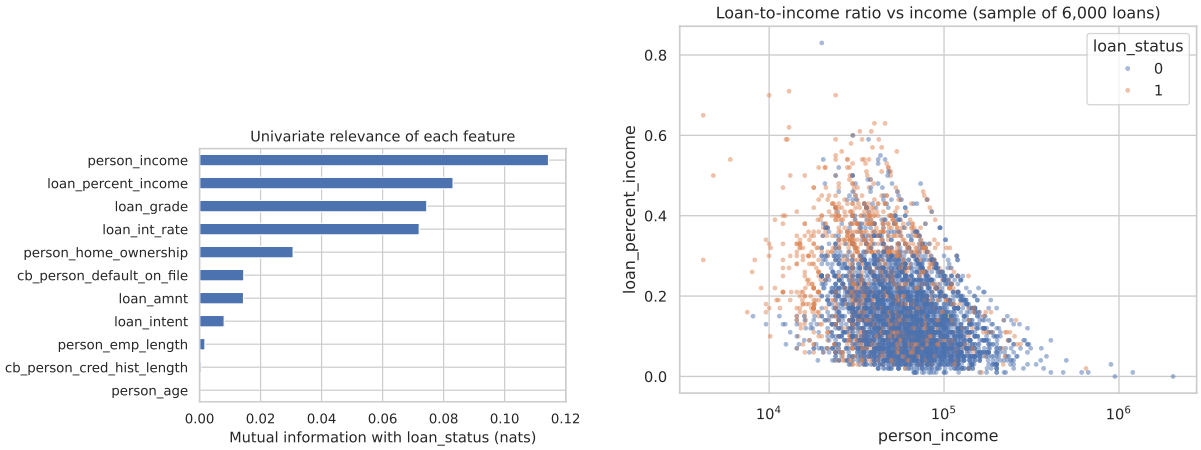


Figure 2.5: Spearman correlations between numeric features, the ordinal grade, the past-default flag and the target.



(a) Mutual information with the target.

(b) Loan-to-income ratio versus income.

Figure 2.6: Relevance of individual features and an example interaction.

2.7 Missing values

Two columns have missing values: `person_emp_length` (895 rows, 2.75%) and `loan_int_rate` (3116 rows, 9.56%). They are missing together in only 68 rows. Figure 2.7 shows that the two columns behave differently.

- A missing **interest rate** is uninformative. Loans with a missing rate default at 20.7%, against 21.9% for loans with an observed rate, and the share of missing rates is flat across grades. This is consistent with *missing completely at random*. Because the rate is almost determined by the grade (fig. 2.8), the grade-wise median is a natural imputation.

- A missing **employment length** is informative. These applicants default at 31.5%, against 21.5% otherwise. The missingness itself carries signal, which motivates the optional missing-value indicators.

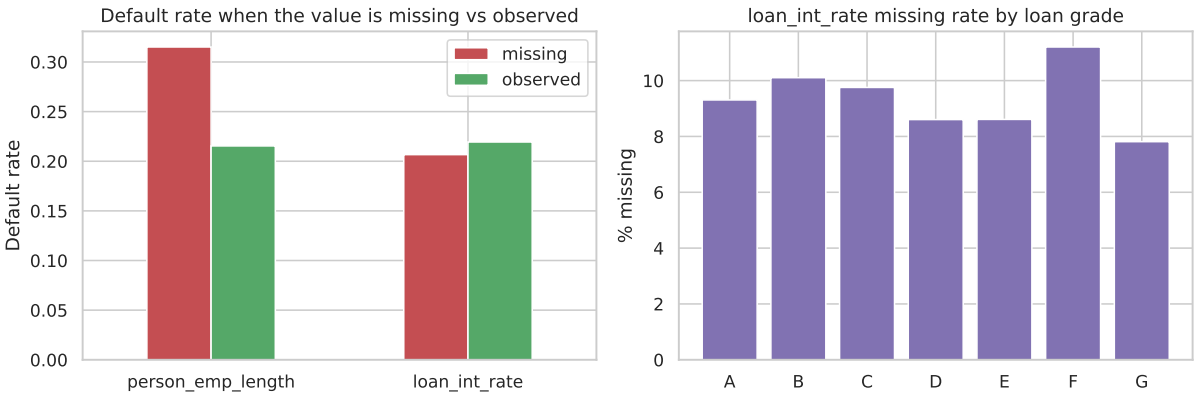


Figure 2.7: Left: default rate when a value is missing or observed. Right: missing share of `loan_int_rate` per grade.

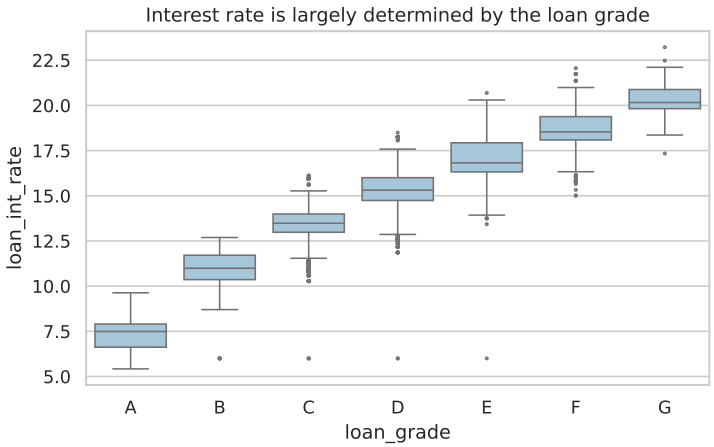


Figure 2.8: Interest rate by loan grade: the grade almost determines the rate.

2.8 Implications for the pipeline

Table 2.4 summarizes how the findings shape the pipeline.

Finding	**Pipeline decision**
165 duplicates, 7 impossible values	Rule-based cleaning before the split; impossible values become missing
21.8% defaults, asymmetric error costs	Stratification; PR-AUC for selection and an F2-tuned threshold; class-imbalance remedies compared
Interest rate determined by grade	Grade-wise median imputation as an option
Informative missing employment length	Optional missing-value indicators
Heavy right tails (income, amounts)	IQR winsorizing or log transform as options; log features in feature engineering
Monotone default rate across grades	Ordinal versus one-hot grade encoding compared
Very different scales (income vs ratios)	Scaler choice compared, essential for k-NN and SVM
Correlated features, only 11 inputs	PCA and feature selection tested, with little redundancy expected
Interactions (income × burden, grade × housing)	Tree ensembles and neural networks expected to do well

Table 2.4: From EDA findings to pipeline options.

3 Methodology

This section describes the evaluation protocol shared by all experiments and then the three modelling approaches. Figure 3.1 gives an overview of how the data flows through the project.

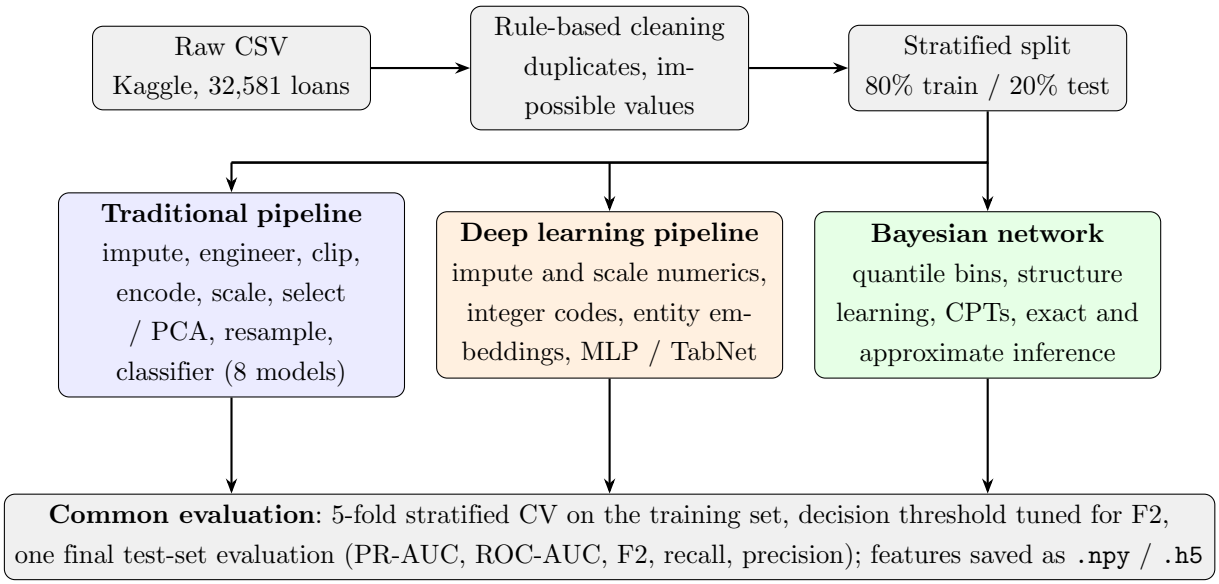


Figure 3.1: Overview of the project: one cleaned and split dataset feeds three model families that share the same evaluation protocol.

3.1 Evaluation protocol

Data split. After rule-based cleaning (Section 2.2), the data are split once into a stratified training set (80%) and test set (20%) with a fixed seed. Every learned step (imputation statistics, scaling parameters, PCA axes, discretization bins, network weights, CPTs, decision thresholds) is estimated on training data only. The test set is used once per final model.

Cross-validation. Model selection uses 5-fold stratified cross-validation on the training set. Because every preprocessing step lives inside one pipeline object, it is refitted on each training fold, so the validation fold never leaks into the preprocessing statistics.

Metrics. Let TP, FP, FN, TN count the outcomes for the positive class (default). We report

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}, \quad F_\beta = \frac{(1 + \beta^2)\,\text{precision} \cdot \text{recall}}{\beta^2\,\text{precision} + \text{recall}}, \tag{3.1}$$

together with accuracy, the area under the ROC curve (ROC-AUC) and the area under the precision-recall curve (PR-AUC, computed as average precision).

Why PR-AUC and F2 instead of accuracy or recall alone. Defaults make up only 21.8% of the loans, so the trivial "never default" rule already reaches 78% accuracy. For a lender, a missed defaulter (false negative) loses the principal, whereas a rejected good applicant (false positive) loses only the interest margin. Recall on the default class is therefore the quantity of interest, but recall alone is degenerate: flagging every applicant gives recall 1. We consequently

1. rank models and select hyper-parameters by **PR-AUC**, a threshold-free measure that, unlike ROC-AUC, is sensitive to performance on the minority class[23]; and
2. choose each model's operating threshold to maximize **F2** ($\beta = 2$ in eq. (3.1)), which weights recall twice as much as precision[28].

The threshold is tuned by internal cross-validation on the training set (`TunedThresholdClassifierCV`) for the traditional models, and on a held-out 15% validation split of the training set for the neural networks and Bayesian networks.

3.2 Traditional machine learning pipeline

The pipeline is assembled by a single function from a declarative `PipelineConfig`, shown in listing 1. All components are standard scikit-learn[20] or imbalanced-learn[17] classes, except two small transformers that have no library equivalent: grade-wise median imputation and IQR clipping.

```python
@dataclass(frozen=True)
class PipelineConfig:
    imputer: str = "median"          # mean | median | group_median | knn | iterative
    add_indicator: bool = False      # append missing-value indicator columns
    feature_engineering: bool = False
    outlier: str = "none"            # none | clip_iqr | log
    grade_encoding: str = "ordinal"  # ordinal | onehot
    scaler: str = "standard"         # none | standard | minmax | robust
    feature_range: tuple = (0.0, 1.0)  # used by MinMaxScaler
    select_k: int | None = None      # SelectKBest(mutual information)
    pca_variance: float | None = None  # e.g. 0.90, 0.95
    resampler: str = "none"          # none | smote | undersample
    class_weight: str | None = None  # None | "balanced"
    model: str = "logreg"            # one of the 8 classifiers below
    model_params: dict = field(default_factory=dict)

pipe = build_pipeline(PipelineConfig(imputer="group_median", scaler="minmax",
                                     feature_range=(-1, 1), pca_variance=0.95,
                                     model="svm"))
```

Listing 1: Every pipeline step is selected through one configuration object.

3.2.1 Missing-value imputation

Five strategies are available. *Mean* and *median* imputation replace a missing value by a training statistic of its column. *Grade-wise median* imputation fills `loan_int_rate` with the median rate of the applicant's loan grade, which the EDA shows to be the main driver of the rate (fig. 2.8). *KNN* imputation [26] averages the five nearest neighbours in standardized feature space. *Iterative* imputation (MICE) [27] regresses each incomplete column on the others in round-robin fashion. Optionally, binary missing-value indicators are appended so that the model can exploit informative missingness [21].

3.2.2 Feature engineering and outliers

Four domain features can be added: $\log(1+\text{income})$, $\log(1+\text{amount})$, income per year of credit history, and the product of the interest rate and the loan-to-income ratio, which approximates the repayment burden. Outliers are handled by IQR winsorizing, which clips every value x of a column to $[Q_1 - 1.5\,\text{IQR},\ Q_3 + 1.5\,\text{IQR}]$ with quartiles learned on the training fold, or by a $\log(1+x)$ transform of all numeric columns.

3.2.3 Encoding and scaling

Nominal variables (`person_home_ownership`, `loan_intent`, `cb_person_default_on_file`) are one-hot encoded. The loan grade has a natural order A $<$ B $<\cdots<$ G, so it can be encoded either ordinally (0–6) or one-hot. Numeric features can be standardized ($z = (x - \mu)/\sigma$), min-max scaled to a configurable range $[a, b]$ via $x' = a + (b - a)(x - x_{\min})/(x_{\max} - x_{\min})$, or robust-scaled with the median and IQR.

3.2.4 Feature selection and dimensionality reduction

`SelectKBest` keeps the k encoded features with the highest mutual information with the target. PCA[13] projects the scaled features onto the leading principal components that together retain a chosen fraction of the variance (80%, 90%, 95% or 99%).

3.2.5 Class imbalance

Three remedies are compared: class weights inversely proportional to class frequency ($w_c = n/(2n_c)$), SMOTE[3], which synthesizes minority samples by interpolating between minority neighbours, and random under-sampling of the majority class. Resampling is applied only to the training folds (it is a pipeline step that is skipped at prediction time).

3.2.6 Classifiers

Table 3.1 lists the eight classifiers. Six of them are suggested by the assignment; Gaussian Naive Bayes and histogram-based gradient boosting are added as a probabilistic baseline and as a strong modern tree ensemble.

Classifier	Decision function / idea
Logistic Regression	$P(y=1 \mid \boldsymbol{x}) = \sigma(\boldsymbol{w}^\top \boldsymbol{x} + b)$, L_2-regularized log-loss with strength $1/C$
Linear SVM	Hinge loss with a linear decision boundary
SVM (RBF)	Maximum-margin classifier with kernel $k(\boldsymbol{x}, \boldsymbol{x}') = \exp(-\gamma \lVert \boldsymbol{x} - \boldsymbol{x}' \rVert^2)$ [5]
k-NN	Majority (or distance-weighted) vote of the k nearest training points
Gaussian Naive Bayes	$P(y \mid \boldsymbol{x}) \propto P(y) \prod_j \mathcal{N}(x_j; \mu_{jy}, \sigma_{jy}^2)$
Decision Tree	Recursive axis-aligned splits minimizing Gini impurity
Random Forest	Bagged decorrelated trees with random feature subsets [2]
Hist. Gradient Boosting	Additive trees fitted to the gradient of the log-loss on binned features [6;14]

Table 3.1: Classifiers available in the traditional pipeline.

3.3 Deep learning pipeline

The deep pipeline replaces hand-made encodings with representations learned end to end (PyTorch [19]). Preprocessing is reduced to what the optimizer needs: numeric columns are imputed, clipped and standardized, and each categorical column is mapped to integer codes, with code 0 reserved for categories unseen during training.

Entity-embedding MLP. Following [11], every categorical variable c with n_c levels owns a trainable embedding table $E_c \in \mathbb{R}^{n_c \times d_c}$ with $d_c = \min(8, \lceil n_c/2 \rceil)$. The embeddings are concatenated with the numeric vector and passed through L blocks of Linear $\to$ BatchNorm [12] $\to$ ReLU $\to$ Dropout [25], followed by a linear output producing a logit z. Training minimizes the class-weighted binary cross-entropy

$$\mathcal{L} = -\frac{1}{n} \sum_{i=1}^{n} \Big[w_+ \, y_i \log \sigma(z_i) + (1 - y_i) \log \big(1 - \sigma(z_i)\big) \Big], \qquad w_+ = \frac{n_0}{n_1}, \tag{3.2}$$

with AdamW [18] (learning rate 2×10^{-3}, weight decay 10^{-4}, batch size 512), a learning rate halved when the validation PR-AUC stalls for three epochs, and early stopping with patience 10 on the validation PR-AUC. Seven variants (table 4.11) vary the width, depth,

dropout, embedding versus one-hot input, class weighting and the engineered features. The 64-dimensional output of the last hidden layer is saved as a learned feature set.

TabNet. TabNet[1] processes the features in several decision steps. At each step a sparse attention mask (sparsemax) selects which features to use, which yields built-in, instance-wise feature importance. We use $n_d = n_a = 16$, four steps, relaxation $\gamma = 1.5$, 4-dimensional categorical embeddings and inverse-frequency sampling against the class imbalance.

3.4 Bayesian network (extension for the talented-engineering program)

A Bayesian network (BN)[15] describes the joint distribution of discrete variables $X_1, \ldots, X_n$ with two ingredients. A directed acyclic graph G encodes the conditional independences, and each node carries a conditional probability table (CPT) given its parents:

$$P(X_1, \ldots, X_n) = \prod_{i=1}^{n} P\big(X_i \mid \mathrm{Pa}_G(X_i)\big). \tag{3.3}$$

Everything below is implemented from scratch with NumPy in `modules/creditrisk/bayesnet/`. Table 3.2 maps the components to files.

File	Content
`graph.py`	DAG with cycle detection on insertion, Kahn's topological sort, Markov blanket
`factor.py`	Discrete factor with product, marginalization (sum-out) and evidence reduction
`network.py`	CPT estimation, log-likelihood, vectorized ancestral sampling
`structure.py`	Naive Bayes, TAN (Chow-Liu), BIC hill climbing, expert DAG
`inference.py`	Variable elimination, enumeration, rejection sampling, likelihood weighting, Gibbs sampling
`discretize.py`	Quantile binning of numeric features learned on the training split

Table 3.2: Bayesian-network modules written for this project.

3.4.1 Discretization and missing values

Each numeric feature is cut into B quantile bins learned on the training split (B is chosen on a validation split, table 4.13). Missing values are *not* imputed: they stay unobserved. During learning, the CPT of X_i is estimated from the rows in which X_i and all its parents are observed (available-case analysis). During prediction, a missing feature is simply left out of the evidence and marginalized by inference, which is the principled treatment of data missing at random.

3.4.2 Parameter learning

With a symmetric Dirichlet(α) prior, the maximum a posteriori CPT entries are smoothed relative frequencies

$$\hat{\theta}_{x|\boldsymbol{u}} = \frac{N(x, \boldsymbol{u}) + \alpha}{N(\boldsymbol{u}) + \alpha\, r_i}, \tag{3.4}$$

where $N(\cdot)$ are counts in the training data and r_i is the number of states of X_i. $\alpha = 1$ is Laplace smoothing; the sensitivity to α is studied in table 4.13.

3.4.3 Structures

Four structures are compared.

- **Naive Bayes**: the class is the only parent of every feature.
- **TAN** (tree-augmented Naive Bayes)[7]: Naive Bayes plus a maximum-weight spanning tree over the features, where the weight of the pair (X_i, X_j) is the conditional mutual information $I(X_i; X_j \mid C)$[4]. The tree is built with Prim's algorithm.
- **Hill climbing on BIC**: greedy search over single-edge additions, deletions and reversals that keep the graph acyclic, with at most three parents per node. It maximizes

$$\mathrm{BIC}(G) = \sum_i \Big[\sum_{x,\boldsymbol{u}} N(x, \boldsymbol{u}) \log \frac{N(x, \boldsymbol{u})}{N(\boldsymbol{u})} - \frac{\log N_i}{2}\, q_i (r_i - 1) \Big], \tag{3.5}$$

  where q_i is the number of parent configurations[24]. Because the score decomposes over families, each move is re-scored locally and family scores are cached.
- **Expert DAG**: a causal story of a loan application written before inspecting the learned graphs. Age drives employment length, credit history and income. Income and amount determine the loan-to-income ratio. Past defaults determine the grade, which sets the interest rate. Default depends on the grade, the repayment burden, housing and the loan purpose.

3.4.4 Exact inference: variable elimination

Any query $P(Q \mid \boldsymbol{e})$ is answered by summing hidden variables out of the product of reduced CPT factors (algorithm 3.1)[29]. Two optimizations matter. First, variables that are not ancestors of Q or $\boldsymbol{e}$ are *barren*: their factors sum to one and are dropped before elimination. Second, the elimination order is chosen greedily by the min-weight heuristic, which eliminates next the variable whose intermediate factor would be smallest. For correctness, the unit tests compare variable elimination with brute-force enumeration of the full joint on random networks and require agreement to 10^{-12}.

Algorithm 3.1: Variable elimination for $P(Q \mid \boldsymbol{e})$

Input: network $(G, \{\theta_i\})$, query variable Q, evidence $\boldsymbol{e}$

$R \leftarrow$ ancestors of $\{Q\} \cup \text{vars}(\boldsymbol{e})$ in G ; // prune barren nodes

$\mathcal{F} \leftarrow \{\, \phi_i = P(X_i \mid \text{Pa}(X_i))[\boldsymbol{e}] : X_i \in R \,\}$; // reduce by evidence

$H \leftarrow R \setminus (\{Q\} \cup \text{vars}(\boldsymbol{e}))$;

for $Z \in \text{MinWeightOrder}(\mathcal{F}, H)$ **do**

  $\mathcal{F}_Z \leftarrow \{\phi \in \mathcal{F} : Z \in \text{scope}(\phi)\}$;

  $\tau \leftarrow \sum_z \prod_{\phi \in \mathcal{F}_Z} \phi$;

  $\mathcal{F} \leftarrow (\mathcal{F} \setminus \mathcal{F}_Z) \cup \{\tau\}$;

return normalize$\big(\prod_{\phi \in \mathcal{F}} \phi\big)$;

3.4.5 Approximate inference

Three sampling algorithms are implemented and vectorized over samples[15;22].

- **Rejection sampling** draws N samples in topological order from eq. (3.3) and keeps those consistent with $\boldsymbol{e}$. The acceptance rate equals $P(\boldsymbol{e})$, which shrinks exponentially with the number of evidence variables.
- **Likelihood weighting**[8] clamps the evidence variables, samples the others and weights each sample by $w = \prod_{E_j \in \boldsymbol{e}} P(e_j \mid \text{Pa}(E_j))$; the estimate is $\hat{P}(q \mid \boldsymbol{e}) = \sum_s w_s \mathbb{1}[q_s = q] / \sum_s w_s$ (algorithm 3.2).
- **Gibbs sampling**[9] runs 20 parallel Markov chains, initialized by likelihood weighting, that resample each non-evidence variable from its distribution given its Markov blanket, $P(x_i \mid \text{mb}(X_i)) \propto P(x_i \mid \text{Pa}(X_i)) \prod_{C \in \text{Ch}(X_i)} P(c \mid \text{Pa}(C))$, and discards a burn-in of 100 sweeps.

Algorithm 3.2: Likelihood weighting for $P(Q \mid \boldsymbol{e})$

Input: network, query Q, evidence $\boldsymbol{e}$, number of samples N

$W[q] \leftarrow 0$ for every state q of Q;
for $s = 1, \ldots, N$ **do**
  $w \leftarrow 1$;
  for X_i *in topological order* **do**
    if $X_i \in \text{vars}(\boldsymbol{e})$ **then**
      $x_i \leftarrow e_i$; $w \leftarrow w \cdot P(e_i \mid \text{pa}_i)$;
    else
      sample $x_i \sim P(X_i \mid \text{pa}_i)$;
  $W[x_Q] \leftarrow W[x_Q] + w$;
return $W / \sum_q W[q]$;

3.4.6 Classification with a Bayesian network

For every test applicant, the network computes $P(\text{default} \mid \text{observed features})$ exactly by variable elimination. Rows with identical evidence share one inference call. The decision threshold is tuned for F2 on a validation split, exactly as for the other model families, so that all three families are compared under the same rule.

4 Experiments and results

4.1 Setup

All experiments run in the Colab notebook with scikit-learn 1.9, imbalanced-learn 0.14, PyTorch 2.14 and pytorch-tabnet 4.1, with every random seed fixed to 42. The deep networks are the exception: each is trained with the five seeds 42–46 and reported as mean ± standard deviation over them. Their training is deterministic for a given seed on one machine, but different CPUs and PyTorch builds round slightly differently, which can move the epoch at which early stopping halts and shift the test scores slightly, most visibly the balance of recall and precision at the tuned threshold; the spread over seeds covers this variation, whereas a single-seed number would not reproduce on other hardware. The complete notebook runs in about 35 minutes on two modern CPU cores (the multi-seed deep models take about 7 of them) and in up to about 2.5 hours on a free Colab runtime; the training times in the tables refer to the former. After cleaning, the 32 416 loans are

split into 25 932 training and 6484 test loans, both with a 21.8% default rate. Unless stated otherwise, scores are means over 5 stratified cross-validation folds of the training set, and "sd" is the standard deviation across folds. In the ablation tables, F2, recall and precision are measured at the default threshold of 0.5, while PR-AUC and ROC-AUC do not depend on a threshold.

4.2 Handling impossible values

Table 4.1 compares six ways of treating the seven impossible ages and employment lengths. All of them lie within about 0.001 PR-AUC of each other, far below the fold-to-fold standard deviation, which is expected because only seven training rows (and no test row) are affected. Since performance cannot separate the options, the choice rests on methodology. Turning the values into missing values and letting the pipeline impute them keeps the other ten valid columns of each row, learns the fill value inside each training fold (no leakage), works with any configured imputer, and is the only option that also applies to a new applicant at prediction time, where a row cannot be dropped.

strategy	Hist. Gradient Boosting	Logistic Regression
1. keep as is	0.89660	0.75358
2. drop rows	0.89681	0.75360
3. replace by mean	0.89769	0.75360
4. replace by median	0.89769	0.75356
5. cap to plausible value	0.89656	0.75360
6. missing, then pipeline imputer	0.89769	0.75356

Table 4.1: Handling the 7 impossible training values: 5-fold CV PR-AUC on identical validation rows (mean fold standard deviation 0.0052).

4.3 Ablation studies of the traditional pipeline

Each suite changes one step of the pipeline and keeps the others at a reference configuration: logistic regression with balanced class weights and standard scaling, k-NN with standard scaling, or random forest without scaling.

4.3.1 Imputation

Table 4.2 shows that the five imputers are practically indistinguishable: all differences are below 0.001 PR-AUC for logistic regression and below 0.0015 for the random forest, well inside one standard deviation. With only 2.75% and 9.56% of the values missing in two columns, the imputation method has little room to matter. The grade-wise median is the best or tied for best for both models and is much cheaper than KNN or MICE, so it is kept. Missing-value indicators help logistic regression slightly (+0.0014) but not the forest, consistent with the informative missingness of the employment length found in section 2.7.

model	imputer	indicator	pr_auc	sd	roc_auc	f2	recall	precision
Logistic Regression	mean	False	0.6984	0.0046	0.8642	0.7061	0.7799	0.5122
Logistic Regression	median	False	0.6985	0.0045	0.8642	0.7063	0.7801	0.5125
Logistic Regression	group_median	False	0.6989	0.0044	0.8643	0.7075	0.7806	0.5149
Logistic Regression	knn	False	0.6986	0.0045	0.8642	0.7059	0.7799	0.5117
Logistic Regression	iterative	False	0.6985	0.0046	0.8642	0.7061	0.7799	0.5122
Logistic Regression	median	True	0.6999	0.0052	0.8652	0.7079	0.7813	0.5145
Random Forest	mean	False	0.8835	0.0016	0.9324	0.7675	0.7381	0.9127
Random Forest	median	False	0.8827	0.0016	0.9313	0.7685	0.7392	0.9136
Random Forest	group_median	False	0.8838	0.0017	0.9325	0.7691	0.7397	0.9144
Random Forest	knn	False	0.8839	0.0016	0.9321	0.7682	0.7380	0.9186
Random Forest	iterative	False	0.8836	0.0011	0.9322	0.7675	0.7376	0.9162
Random Forest	median	True	0.8824	0.0027	0.9316	0.7690	0.7404	0.9097

Table 4.2: Imputation strategies: 5-fold CV on the training set (F2, recall and precision at threshold 0.5).

4.3.2 Scaling

Scaling is the step with the largest effect (table 4.3 and fig. 4.1). Without it, the RBF kernel and the Euclidean distance of k-NN are dominated by income, whose values are four orders of magnitude larger than the ratios. Standardization raises the k-NN PR-AUC from 0.615 to 0.778. The unscaled SVM was too slow to converge on two cores and was excluded; among the scaled variants, the SVM prefers standardization (0.834) and robust scaling (0.830) to min-max scaling (0.797 and 0.804). Min-max scaling squeezes the bulk of the skewed features into a tiny interval because the bounds are set by the extremes. For logistic regression, every scaler gives the same result (0.699 to 0.700) because the solution of a regularized linear model changes only slightly with the feature scale, but leaving the features unscaled hurts (0.665) because the regularization then penalizes the coefficients

unevenly and the solver converges poorly.

model	scaler	pr_auc	sd	roc_auc	f2	recall	precision
Logistic Regression	none	0.6650	0.0098	0.8449	0.6884	0.7792	0.4696
Logistic Regression	standard	0.6985	0.0045	0.8642	0.7063	0.7801	0.5125
Logistic Regression	minmax [0,1]	0.6996	0.0048	0.8642	0.7066	0.7810	0.5120
Logistic Regression	minmax [-1,1]	0.6990	0.0049	0.8642	0.7069	0.7808	0.5129
Logistic Regression	robust	0.6986	0.0046	0.8642	0.7060	0.7798	0.5124
k-NN	none	0.6153	0.0109	0.8203	0.4926	0.4594	0.6949
k-NN	standard	0.7775	0.0072	0.8827	0.6114	0.5664	0.8964
k-NN	minmax [0,1]	0.7720	0.0072	0.8778	0.6098	0.5646	0.8972
k-NN	minmax [-1,1]	0.7723	0.0071	0.8779	0.6113	0.5662	0.8975
k-NN	robust	0.7708	0.0070	0.8820	0.5766	0.5288	0.9026
SVM (RBF)	standard	0.8340	0.0079	0.9075	0.7543	0.7674	0.7063
SVM (RBF)	minmax [0,1]	0.7966	0.0068	0.8967	0.7365	0.7644	0.6426
SVM (RBF)	minmax [-1,1]	0.8040	0.0071	0.9008	0.7433	0.7711	0.6496
SVM (RBF)	robust	0.8302	0.0082	0.9090	0.7581	0.7768	0.6918

Table 4.3: Scaling methods: 5-fold CV on the training set (F2, recall and precision at threshold 0.5).

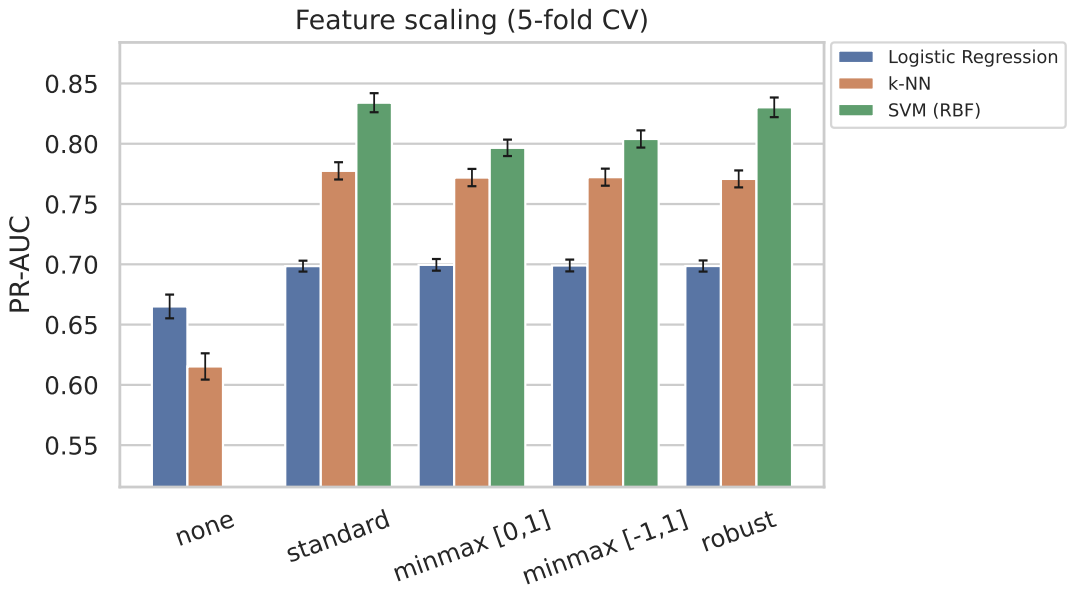


Figure 4.1: Effect of the scaler on PR-AUC (5-fold CV; error bars show one standard deviation).

4.3.3 Outliers and feature engineering

Table 4.4 crosses the three outlier treatments with the optional engineered features. For logistic regression, IQR clipping combined with feature engineering is the best configuration (0.722 against 0.699 for the baseline, a gain of about five standard deviations). The log transform alone beats clipping alone, but once the engineered features already provide log-income and log-amount, clipping wins: it removes the leverage of the remaining extreme raw values without bending the whole distribution. For the random forest, the three treatments differ by less than 0.0005, because a tree splits on thresholds and any monotone transformation leaves its splits unchanged; feature engineering adds a small gain (+0.0025).

Removing outliers instead of clipping them was rejected for a measurable reason: 17.2% of the training rows contain at least one IQR outlier, and 75.9% of the rows whose loan-to-income ratio is an outlier defaulted. Deleting them would discard precisely the most informative defaulters, and a deployed model cannot refuse to score such applicants anyway.

model	outlier	feat_eng	pr_auc	sd	roc_auc	f2	recall	precision
Logistic Regression	none	False	0.6985	0.0045	0.8642	0.7063	0.7801	0.5125
Logistic Regression	none	True	0.7122	0.0037	0.8761	0.7221	0.7983	0.5226
Logistic Regression	clip_iqr	False	0.7021	0.0049	0.8656	0.7082	0.7833	0.5120
Logistic Regression	clip_iqr	True	0.7220	0.0053	0.8771	0.7273	0.8022	0.5297
Logistic Regression	log	False	0.7105	0.0040	0.8717	0.7133	0.7877	0.5179
Logistic Regression	log	True	0.7160	0.0035	0.8765	0.7227	0.7958	0.5285
Random Forest	none	False	0.8827	0.0016	0.9313	0.7685	0.7392	0.9136
Random Forest	none	True	0.8852	0.0011	0.9332	0.7721	0.7466	0.8942
Random Forest	clip_iqr	False	0.8834	0.0020	0.9319	0.7690	0.7396	0.9146
Random Forest	clip_iqr	True	0.8854	0.0011	0.9333	0.7718	0.7461	0.8951
Random Forest	log	False	0.8825	0.0018	0.9312	0.7685	0.7390	0.9143
Random Forest	log	True	0.8852	0.0011	0.9331	0.7721	0.7466	0.8942

Table 4.4: Outlier handling and feature engineering: 5-fold CV on the training set (F2, recall and precision at threshold 0.5).

4.3.4 Encoding of the loan grade

Table 4.5 shows that one-hot encoding of the grade helps logistic regression markedly (0.717 against 0.699). The ordinal code forces a linear effect per grade step, whereas the default rate jumps between grades C and D (fig. 2.4). Tree models and k-NN are slightly better with the compact ordinal code, which preserves the order and lets a tree place the

split between C and D itself.

model	grade_encoding	pr_auc	sd	roc_auc	f2	recall	precision
Logistic Regression	ordinal	0.6985	0.0045	0.8642	0.7063	0.7801	0.5125
Logistic Regression	onehot	0.7165	0.0061	0.8708	0.7181	0.7791	0.5470
k-NN	ordinal	0.7775	0.0072	0.8827	0.6114	0.5664	0.8964
k-NN	onehot	0.7748	0.0083	0.8787	0.6179	0.5743	0.8871
Random Forest	ordinal	0.8827	0.0016	0.9313	0.7685	0.7392	0.9136
Random Forest	onehot	0.8819	0.0018	0.9309	0.7667	0.7392	0.9010

Table 4.5: Encoding of the loan grade: 5-fold CV on the training set (F2, recall and precision at threshold 0.5).

4.3.5 Dimensionality reduction and feature selection

PCA lowers the PR-AUC of every model as soon as it discards variance (table 4.6 and fig. 4.2), and the loss grows as fewer components are kept; at 99% it keeps almost every dimension and is merely neutral. The damage is largest for the random forest (from 0.883 to 0.766 at 80%), whose axis-aligned splits lose the original, individually meaningful features when they are rotated into components. Univariate selection with mutual information hurts logistic regression and the random forest, because it ranks features one at a time and discards those that matter mainly in interaction with others. For k-NN, keeping the 15 most informative features gives a gain of 0.005 that is within one standard deviation, so no reliable improvement either. With about twenty encoded features and only moderate redundancy, there is nothing to gain from reducing the dimension, so neither step is used in the final pipelines.

model	reduction	pr_auc	sd	roc_auc	f2	recall	precision
Logistic Regression	none	0.6985	0.0045	0.8642	0.7063	0.7801	0.5125
Logistic Regression	PCA 80%	0.6485	0.0095	0.8391	0.6867	0.7755	0.4711
Logistic Regression	PCA 90%	0.6571	0.0048	0.8447	0.6869	0.7685	0.4824
Logistic Regression	PCA 95%	0.6755	0.0086	0.8517	0.6957	0.7764	0.4915
Logistic Regression	PCA 99%	0.6983	0.0046	0.8642	0.7064	0.7805	0.5123
Logistic Regression	SelectKBest k=5	0.6693	0.0055	0.8475	0.6984	0.7798	0.4927
Logistic Regression	SelectKBest k=10	0.6837	0.0053	0.8572	0.7007	0.7745	0.5075
Logistic Regression	SelectKBest k=15	0.6920	0.0059	0.8614	0.7009	0.7739	0.5090
k-NN	none	0.7775	0.0072	0.8827	0.6114	0.5664	0.8964
k-NN	PCA 80%	0.7251	0.0217	0.8648	0.5695	0.5317	0.7960
k-NN	PCA 90%	0.7462	0.0092	0.8732	0.5939	0.5544	0.8304
k-NN	PCA 95%	0.7550	0.0099	0.8755	0.5954	0.5528	0.8610
k-NN	PCA 99%	0.7776	0.0070	0.8827	0.6112	0.5662	0.8958
k-NN	SelectKBest k=5	0.7592	0.0045	0.8731	0.6281	0.5943	0.8136
k-NN	SelectKBest k=10	0.7695	0.0077	0.8841	0.6429	0.6105	0.8170
k-NN	SelectKBest k=15	0.7823	0.0110	0.8851	0.6258	0.5828	0.8890
Random Forest	none	0.8826	0.0017	0.9313	0.7677	0.7383	0.9133
Random Forest	PCA 80%	0.7664	0.0296	0.8822	0.6810	0.6794	0.6880
Random Forest	PCA 90%	0.8196	0.0074	0.9041	0.7096	0.6978	0.7612
Random Forest	PCA 95%	0.8280	0.0073	0.9072	0.7148	0.7002	0.7800
Random Forest	PCA 99%	0.8363	0.0076	0.9100	0.7300	0.7156	0.7945
Random Forest	SelectKBest k=5	0.8301	0.0047	0.9053	0.7376	0.7374	0.7385
Random Forest	SelectKBest k=10	0.8355	0.0068	0.9122	0.7463	0.7491	0.7359
Random Forest	SelectKBest k=15	0.8684	0.0045	0.9253	0.7553	0.7369	0.8400

Table 4.6: PCA and univariate feature selection: 5-fold CV on the training set (F2, recall and precision at threshold 0.5).

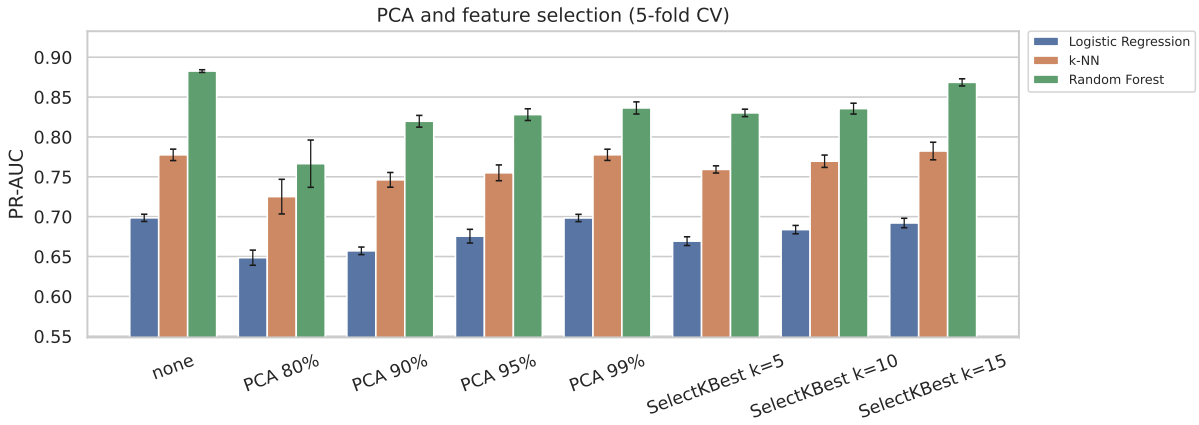


Figure 4.2: PCA and feature selection lower PR-AUC for every model (5-fold CV).

4.3.6 Class imbalance

The imbalance strategies barely change PR-AUC (table 4.7), which is expected: re-weighting or resampling mostly shifts the predicted probabilities, and PR-AUC ignores the threshold. They change the operating point instead. At the default threshold of 0.5, logistic regression without any remedy finds only half of the defaulters (recall 0.505), and class weights raise its recall to 0.780 and its F2 from 0.539 to 0.706. For the gradient-boosting model, class weights give the best F2 (0.792) with the best PR-AUC. SMOTE never beats class weights, and random under-sampling costs PR-AUC for the tree models because it throws away three quarters of the non-default examples. Because the final models tune their decision threshold for F2 anyway (section 3.1), class weighting becomes a hyper-parameter of the grid search rather than a fixed choice.

model	imbalance	pr_auc	sd	roc_auc	f2	recall	precision
Logistic Regression	none	0.7026	0.0039	0.8637	0.5390	0.5050	0.7381
Logistic Regression	class_weight	0.6985	0.0045	0.8642	0.7063	0.7801	0.5125
Logistic Regression	smote	0.6983	0.0050	0.8638	0.7041	0.7750	0.5155
Logistic Regression	undersample	0.6983	0.0038	0.8641	0.7080	0.7822	0.5133
Random Forest	none	0.8843	0.0017	0.9314	0.7554	0.7152	0.9743
Random Forest	class_weight	0.8827	0.0016	0.9313	0.7685	0.7392	0.9136
Random Forest	smote	0.8792	0.0023	0.9280	0.7568	0.7182	0.9641
Random Forest	undersample	0.8766	0.0016	0.9291	0.7809	0.7939	0.7334
Hist. Gradient Boosting	none	0.8982	0.0016	0.9440	0.7657	0.7265	0.9761
Hist. Gradient Boosting	class_weight	0.8986	0.0029	0.9442	0.7923	0.7819	0.8370
Hist. Gradient Boosting	smote	0.8974	0.0016	0.9428	0.7678	0.7288	0.9766
Hist. Gradient Boosting	undersample	0.8928	0.0023	0.9406	0.7921	0.7962	0.7762

Table 4.7: Class-imbalance strategies: 5-fold CV on the training set (F2, recall and precision at threshold 0.5).

4.4 Tuned comparison of the traditional models

Following the ablations, the scale-sensitive models use grade-wise imputation, engineered features, IQR clipping, a one-hot grade and standardization, and the tree models use grade-wise imputation, engineered features and the ordinal grade without scaling. Table 4.8 lists the grid-search results and table 4.9 the test-set results at the F2-tuned threshold.

model	best_params	CV PR-AUC	sd	CV ROC-AUC	CV F2	time (s)
Logistic Regression	C=10, class_weight=None	0.7538	0.0072	0.8827	0.6247	3.6
Linear SVM	C=1, class_weight=None	0.7473	0.0071	0.8801	0.6152	2.4
SVM (RBF)	C=10, gamma=scale	0.8501	0.0045	0.9115	0.7634	164.5
k-NN	n_neighbors=15, weights=distance	0.8134	0.0058	0.8934	0.6459	12.5
Gaussian Naive Bayes	var_smoothing=0.001	0.6229	0.0120	0.8552	0.6077	0.9
Decision Tree	class_weight=None, max_depth=10, min_samples_leaf=20	0.8598	0.0040	0.9151	0.7438	5.6
Random Forest	class_weight=None, max_features=0.5, min_samples_leaf=3	0.8902	0.0025	0.9341	0.7526	198.4
Hist. Gradient Boosting	class_weight=None, learning_rate=0.05, max_iter=300, max_leaf_nodes=31	0.8988	0.0026	0.9441	0.7656	22.1

Table 4.8: Grid-search results (5-fold CV, refit on PR-AUC; F2 at threshold 0.5).

model	pr_auc	roc_auc	threshold	F2	recall	precision	accuracy
Hist. Gradient Boosting	**0.9045**	**0.9459**	0.1515	**0.8132**	**0.8512**	0.6901	0.8839
Random Forest	0.8934	0.9361	0.2222	0.8021	0.8068	0.7841	0.9092
Decision Tree	0.8623	0.9180	0.1414	0.7670	0.7856	0.7006	0.8797
SVM (RBF)	0.8561	0.9147	-0.4900	0.7703	0.8117	0.6398	0.8589
k-NN	0.8163	0.8925	0.2424	0.7501	0.7906	0.6228	0.8495
Logistic Regression	0.7512	0.8797	0.2224	0.7360	0.7955	0.5665	0.8222
Linear SVM	0.7443	0.8774	-0.4521	0.7339	0.7962	0.5589	0.8180
Gaussian Naive Bayes	0.6244	0.8503	0.0101	0.7140	0.8096	0.4850	0.7704

Table 4.9: Test-set results of the tuned traditional models; F2, recall, precision and accuracy at the F2-tuned threshold.

Histogram-based gradient boosting is the best model on every ranking metric, with a test PR-AUC of 0.9045 and ROC-AUC of 0.946. At its F2-tuned threshold it detects 85.1% of the defaulters (recall) with a precision of 69.0%. The random forest follows closely and, at its own threshold, trades some recall for much higher precision. The gap between the tree ensembles and the linear models (PR-AUC 0.75) is large and consistent across folds, which confirms the EDA finding that default risk depends on interactions, for example between the loan-to-income ratio and income, that a linear decision function cannot express. The two SVMs expose a decision function rather than a probability, so their thresholds are margins and can be negative. Gaussian Naive Bayes is the weakest model: its independence and Gaussian assumptions are violated by the strongly correlated,

skewed features. The cross-validation and test PR-AUC agree closely for every model, so the test results are not the product of a lucky split.

Figure 4.3 shows the ROC and precision-recall curves and fig. 4.4 the confusion matrices at the tuned thresholds.

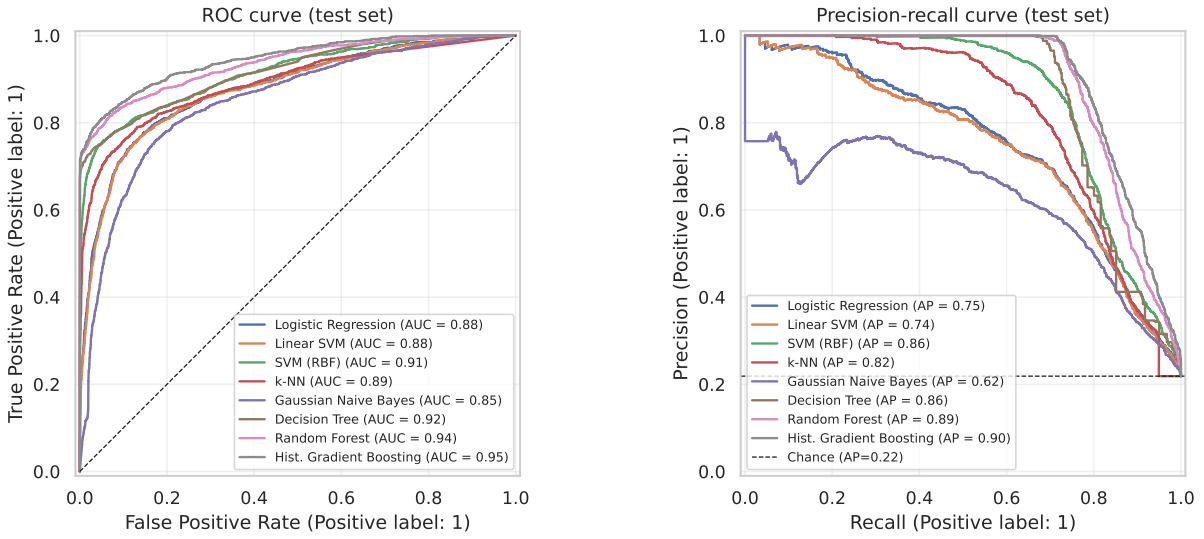


Figure 4.3: ROC (left) and precision-recall (right) curves of the tuned traditional models on the test set.

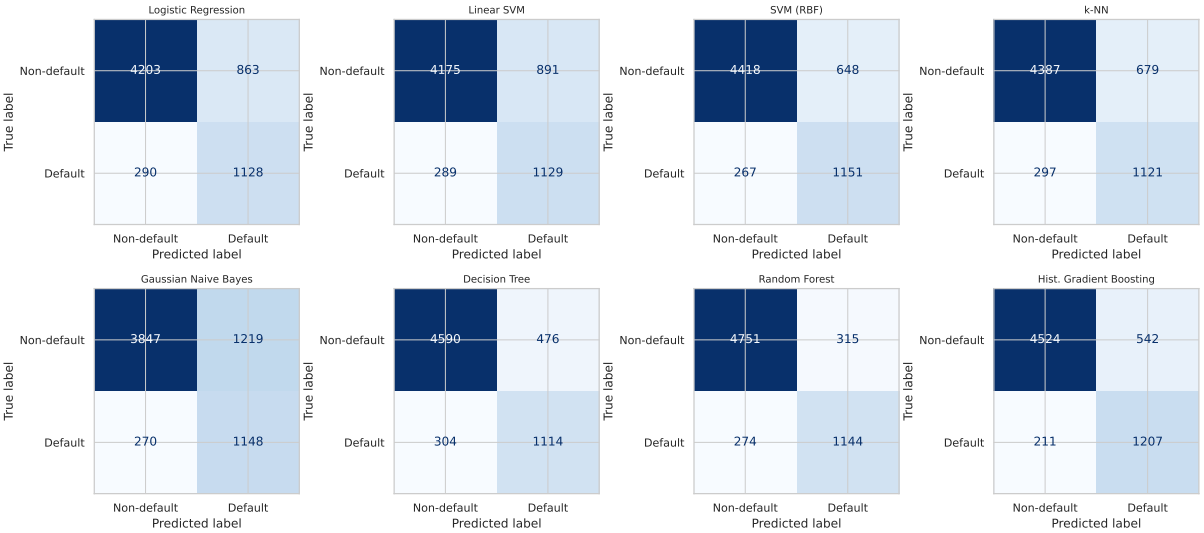


Figure 4.4: Confusion matrices on the test set at each model's F2-tuned threshold.

4.4.1 Is there headroom left?

Several grid optima lie on the edge of their grids (table 4.8), which could indicate a search space that is too narrow. Table 4.10 tests this for the best model family with two stronger optimizers: a random search over seven gradient-boosting hyper-parameters with 40 candidates, and a stacked ensemble of the tuned gradient-boosting, random-forest and logistic-regression models. Neither improves the cross-validated PR-AUC by more than its standard deviation. The model sits on a plateau, so the remaining errors come from the overlap of the classes in feature space rather than from insufficient optimization, and the simpler grid-searched model is kept.

approach	CV PR-AUC	sd	gain vs grid
Hist. Gradient Boosting, grid search (8 candidates)	0.8988	0.0026	0.0000
Hist. Gradient Boosting, random search (40 candidates, 7 hyper-parameters)	0.8986	0.0022	-0.0003
Stacking (Hist. GB + Random Forest + Logistic Regression)	0.9002	0.0032	0.0014

Table 4.10: Headroom check: stronger optimization of the best model family (5-fold CV on the training set).

4.4.2 Feature importance and error analysis

The permutation importance of the best model (fig. 4.5) ranks the loan-to-income ratio, home ownership, income and the loan grade as the most influential features, followed by the loan intent, in line with the mutual-information ranking of the EDA. The interest rate matters little once the grade is known, because the two are almost redundant ($\rho = 0.95$), and age and credit-history length contribute almost nothing. The 211 false negatives at the tuned threshold are dominated by applicants who look safe: 71% of them have grade A or B (against 33% of the true positives), and their median loan-to-income ratio is 0.14 (against 0.27). These borrowers default for reasons that the recorded features do not capture.

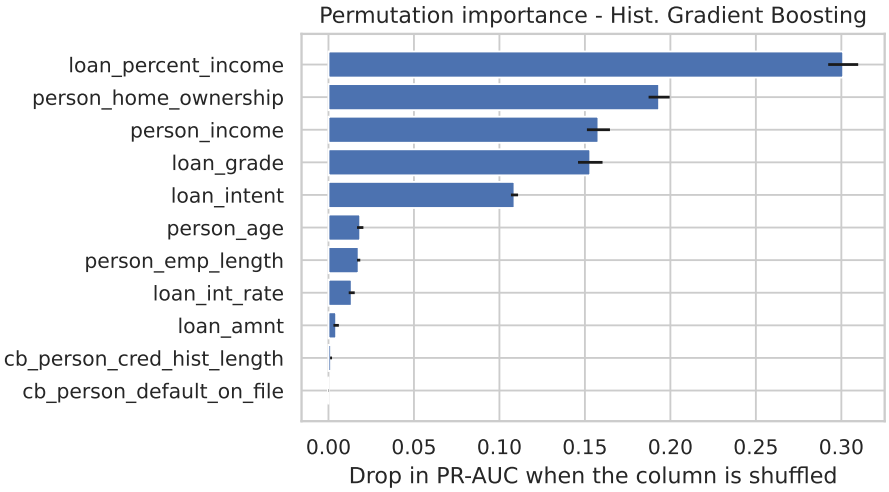


Figure 4.5: Permutation importance of the best traditional model on the test set (drop in PR-AUC).

4.5 Deep learning pipeline

4.5.1 MLP variants

Table 4.11 compares seven MLP variants, each trained with five seeds. The validation PR-AUC, measured on the internal validation split, is the selection criterion; the test columns are reported for reference. The variants lie within a narrow band: the deeper 256-128-64 network is best on both validation and test data, but its lead over the reference network (0.866 against 0.864 on validation) is smaller than the seed-to-seed standard deviation of the validation score (about 0.007), so none of the architectures is clearly better. Entity embeddings and one-hot inputs perform similarly here, because the categorical variables have at most seven levels; embeddings pay off mainly with high-cardinality categories. Removing the class weighting barely changes PR-AUC but lowers recall at the default threshold, mirroring the traditional models, and the engineered features help the network less than they help logistic regression, because the network can learn similar nonlinear transformations itself. Figure 4.6 shows a typical training curve (seed 42): the validation PR-AUC saturates within a few dozen epochs and early stopping restores the best weights.

model	val PR-AUC	pr_auc	roc_auc	F2	recall	epochs	time (s)
MLP 128-64 (reference)	0.8640 ± 0.0082	0.8619 ± 0.0017	0.9200 ± 0.0017	0.7621 ± 0.0017	0.7956 ± 0.0088	56	6.8
MLP 64-32 (smaller)	0.8638 ± 0.0077	0.8623 ± 0.0017	0.9208 ± 0.0014	0.7619 ± 0.0003	0.8017 ± 0.0177	66	5.6
MLP 256-128-64 (deeper)	0.8664 ± 0.0069	0.8650 ± 0.0019	0.9213 ± 0.0015	0.7644 ± 0.0035	0.7949 ± 0.0117	44	9.9
No dropout	0.8596 ± 0.0078	0.8582 ± 0.0022	0.9183 ± 0.0022	0.7624 ± 0.0027	0.7946 ± 0.0116	35	2.6
One-hot instead of embeddings	0.8624 ± 0.0072	0.8609 ± 0.0032	0.9196 ± 0.0014	0.7609 ± 0.0031	0.7946 ± 0.0155	48	5.0
No class weighting	0.8645 ± 0.0063	0.8623 ± 0.0020	0.9191 ± 0.0015	0.7583 ± 0.0037	0.7982 ± 0.0204	51	6.0
No engineered features	0.8618 ± 0.0059	0.8604 ± 0.0004	0.9191 ± 0.0005	0.7578 ± 0.0048	0.8117 ± 0.0300	55	6.5

Table 4.11: MLP variants: validation PR-AUC (used for selection) and test metrics, mean ± sd over 5 seeds (epochs and training time are per-seed means).

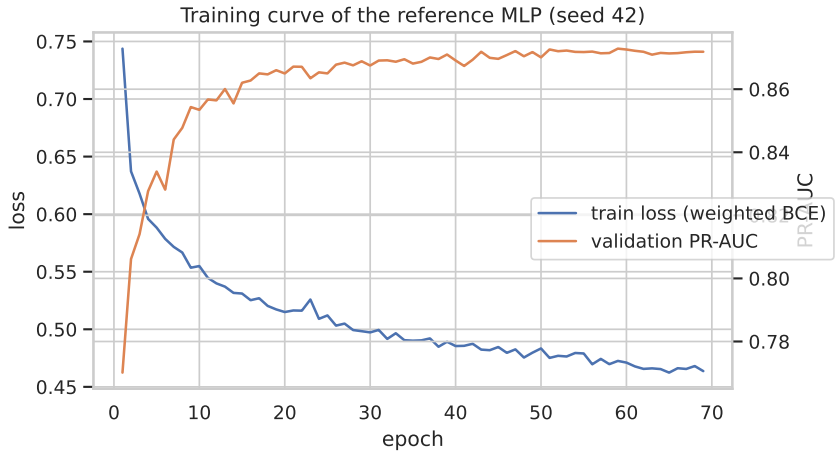


Figure 4.6: Training loss and validation PR-AUC of the reference MLP (seed 42).

4.5.2 TabNet and comparison with the traditional pipeline

Table 4.12 summarizes the deep models on the test set. The reference MLP reaches a PR-AUC of 0.862 ± 0.002, better than every linear model and comparable to a single decision tree (0.862), but clearly below the random forest (0.893) and gradient boosting (0.904). TabNet reaches 0.861 ± 0.003, the same as the MLP within the seed spread, with a similar recall and precision at the tuned threshold; its precision varies more from seed to seed, it takes about five times longer to train and it has more hyper-parameters to tune. Its attention masks for seed 42 (fig. 4.8) agree with the permutation importances on the leading role of the loan-to-income ratio and home ownership. They spread the remaining weight differently: the interest rate takes over much of the grade's information, since the two are strongly correlated, and the credit-history length receives more weight than the other measures give it. This ranking agrees with large tabular benchmarks, in which gradient-boosted trees remain ahead of deep networks on medium-sized tables of heterogeneous features[10]. A logistic regression trained on the 64-dimensional embeddings

of the seed-42 reference MLP reaches a PR-AUC of 0.863, almost exactly that network's own 0.864, which shows that the learned representation, not the final layer, carries the network's advantage over the linear models.

model	pr_auc	roc_auc	F2	recall	precision	accuracy	time (s)
MLP 128-64 (reference)	0.8619 ± 0.0017	0.9200 ± 0.0017	0.7621 ± 0.0017	0.7956 ± 0.0088	0.6535 ± 0.0283	0.8626 ± 0.0105	6.8
TabNet	0.8612 ± 0.0026	0.9180 ± 0.0014	0.7568 ± 0.0039	0.7984 ± 0.0170	0.6288 ± 0.0398	0.8517 ± 0.0148	34.0

Table 4.12: Deep learning models on the test set, mean ± sd over 5 seeds (threshold tuned for F2 on a validation split; training time per seed).

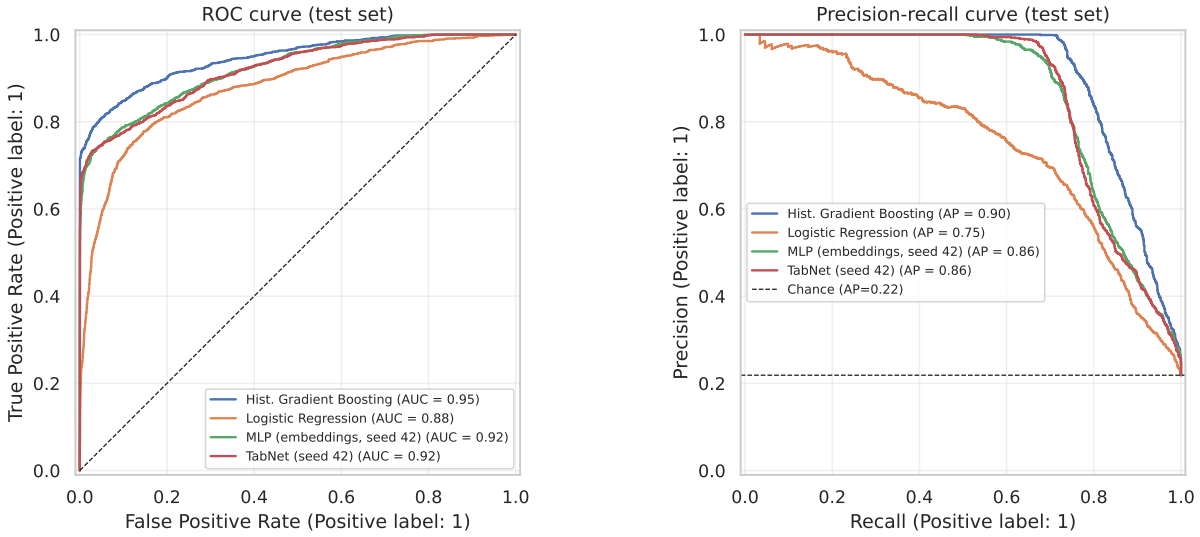


Figure 4.7: Best traditional model, logistic regression, MLP and TabNet (seed 42) on the test set.

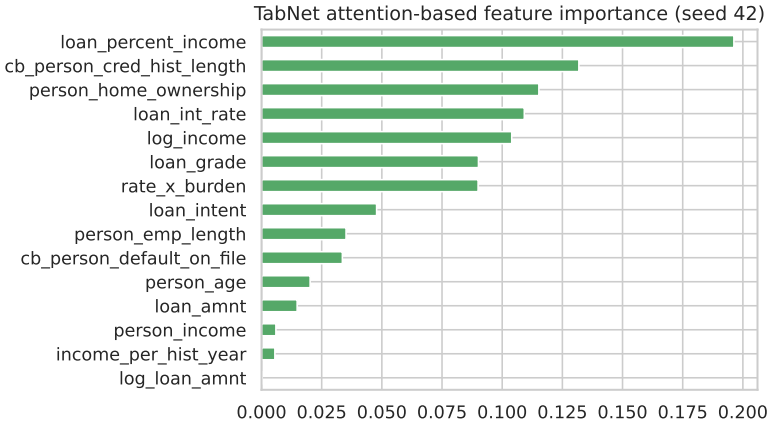


Figure 4.8: TabNet's attention-based feature importance (seed 42).

4.6 Bayesian networks

4.6.1 Discretization and smoothing

The number of quantile bins and the Dirichlet pseudo-count α are chosen separately for each structure on a validation split of the training set (table 4.13). Coarse bins (3 or 5) discard too much information for every structure. Beyond that, the structures disagree: TAN and the expert network peak at 8 bins, while Naive Bayes and hill climbing still improve at 12. The reason is model complexity. A TAN node has the class and another feature as parents, so its CPT grows with the square of the number of bins and starts to overfit sooner, whereas hill climbing is protected by the BIC penalty, which simply removes edges when the CPTs become too large. A small pseudo-count ($\alpha = 0.1$) is preferred by the structures with large CPTs, because a large α pulls the many sparsely observed rows of their tables toward the uniform distribution. An earlier version of the experiment selected a single setting for all structures by averaging; it chose 12 bins and reduced the test PR-AUC of TAN by 0.03, which is why the selection is made per structure.

n_bins	alpha	Expert DAG	Hill-climb BIC	Naive Bayes	TAN
3	0.1	0.7024	0.6088	0.6251	0.7401
3	1	0.7015	0.6111	0.6249	0.7393
3	10	0.6912	0.6114	0.6261	0.7312
5	0.1	0.7338	0.6479	0.6633	0.7672
5	1	0.7316	0.6479	0.6649	0.7656
5	10	0.7111	0.6457	0.6645	0.7578
8	0.1	0.7948	0.6859	0.6905	0.7915
8	1	0.7908	0.6879	0.6919	0.7893
8	10	0.7674	0.6871	0.6916	0.7787
12	0.1	0.7940	0.7416	0.6940	0.7542
12	1	0.7915	0.7416	0.6956	0.7515
12	10	0.7780	0.7383	0.6954	0.7403

Table 4.13: Bayesian networks: validation PR-AUC by number of bins and Dirichlet pseudo-count α.

4.6.2 Structures and classification

Figure 4.9 shows the four structures and table 4.14 their test performance. TAN is the best network classifier, and it is also the only structure whose design targets the prediction of the class. Test log-likelihoods are only comparable between networks that share a discretization. Among the 12-bin networks, hill climbing models the joint distribution far better than Naive Bayes (a log-likelihood of $-103\,692$ against $-136\,946$) yet classifies only slightly better. Among the 8-bin networks, the expert DAG has the higher likelihood but TAN the better classification. BIC rewards a good model of the *whole* joint distribution; most of the learned edges describe relations among the features, and the class ends up with few parents. This reproduces the observation of Friedman et al. [7] that structures selected by a global score are not necessarily good classifiers. The expert network is competitive with TAN despite having only 12 edges, which shows that the domain story captures the main drivers of default. Naive Bayes is the weakest structure, as its conditional-independence assumption is contradicted by the strong correlations among the features.

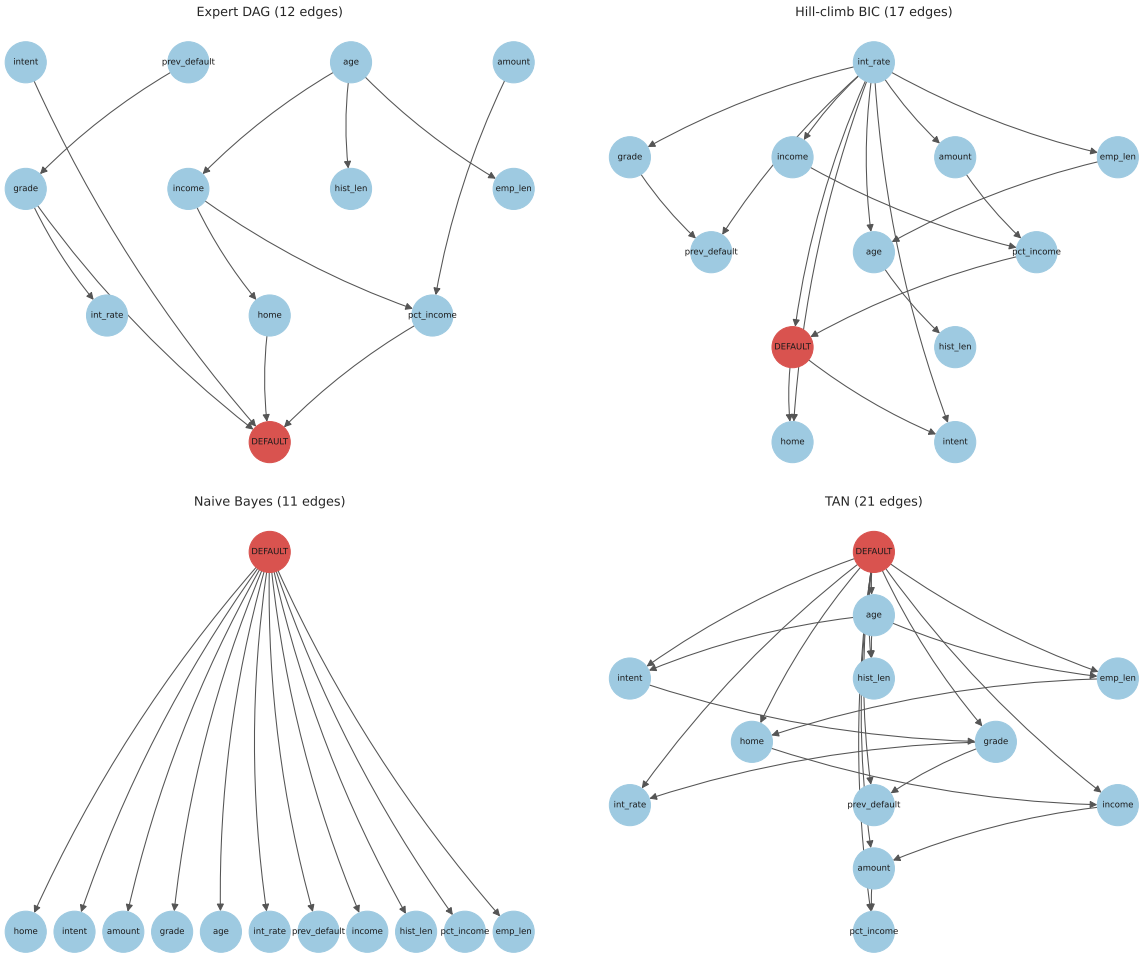


Figure 4.9: The four Bayesian-network structures (DEFAULT is the class node).

structure	bins	alpha	edges	parameters	test log-lik.	pr_auc	roc_auc	F2	recall	precision	accuracy
Expert DAG	8	0.1	12	2041	-100750	0.7822	0.8712	0.7152	0.7736	0.5493	0.8117
Hill-climb BIC	12	0.1	17	3736	-103692	0.7332	0.8579	0.7033	0.7941	0.4826	0.7688
Naive Bayes	12	1	11	171	-136946	0.7061	0.8635	0.7149	0.7962	0.5076	0.7866
TAN	8	0.1	21	777	-105532	**0.7895**	**0.8900**	0.7390	0.8110	0.5453	0.8108

Table 4.14: Bayesian-network structures on the test set, each with the bins and α selected on validation data.

All networks are clearly weaker than the gradient-boosted trees, mainly because discretization throws away within-bin information and because each CPT can only express interactions among a node and its few parents. In return, a Bayesian network handles missing features natively, answers any conditional query, and exposes its assumptions explicitly.

4.6.3 Probabilistic reasoning

Table 4.15 answers predictive and diagnostic queries exactly by variable elimination and compares them with the empirical frequencies in the training data. Because the class is the root of TAN and a parent of every feature, TAN reproduces every single-evidence query almost exactly, both predictive ones such as $P(\text{default} \mid \text{grade} = A)$ and diagnostic ones such as $P(\text{past default} \mid \text{default})$. It fails on the joint query $P(\text{default} \mid \text{grade} = D, \text{home} = \text{OWN})$: TAN answers 0.27 while only 4.6% of these 216 borrowers defaulted. In TAN the grade and the housing status interact only through the tree over the features, so the model cannot represent that owning a home largely cancels the risk of a poor grade. The expert network, in which both are parents of DEFAULT, answers 0.078, much closer to the truth. Conversely, the expert network overestimates the base rate (0.26 against 0.22) and the risk of grade A borrowers, because it assumes, for example, that the grade and the loan-to-income ratio are independent and therefore spreads probability over combinations that rarely occur. These discrepancies are useful diagnostics: each one points to a specific independence assumption that the data contradict.

query	empirical	support	TAN	Expert DAG
P(default)	0.2187	25932	0.2187	0.2594
P(default | grade=A)	0.0998	8584	0.0998	0.1750
P(default | grade=D)	0.5902	2889	0.5900	0.5749
P(default | grade=G)	0.9792	48	0.9675	0.6361
P(default | home=RENT)	0.3167	13114	0.3149	0.3675
P(default | home=OWN)	0.0715	2042	0.0681	0.1696
P(default | grade=D, home=OWN)	0.0463	216	0.2731	0.0780
P(default | intent=MEDICAL)	0.2672	4842	0.2672	0.3008
P(default | intent=VENTURE)	0.1505	4610	0.1506	0.1901
P(prev_default=Y | default=1)	0.3063	5671	0.3064	0.2685
P(prev_default=Y | default=0)	0.1404	20261	0.1404	0.1445
P(grade in D..G | default=1)	0.4195	5671	0.4196	0.3408
P(grade in D..G | default=0)	0.0746	20261	0.0747	0.0832

Table 4.15: Exact answers of two networks (variable elimination) against the empirical frequency in the training data; support is the number of training rows matching the evidence.

4.6.4 Exact versus approximate inference

Exact inference is cheap here: variable elimination answers a query in about 0.1 ms thanks to barren-node pruning and the small width of the networks. Tables 4.16 and 4.17 and fig. 4.10 compare the three sampling algorithms with the exact answers on the TAN network. With two evidence variables, all three methods converge, with an error that shrinks roughly as $1/\sqrt{N}$. Rejection sampling, however, accepts only the samples that match the evidence, and the probability of matching all evidence falls exponentially with the number of evidence variables. With five evidence variables only about 0.1% of the samples are kept, and with ten evidence variables (a typical applicant) not a single sample out of 10 000 matches, so the method returns no answer at all. Likelihood weighting and Gibbs sampling do not have this problem and reach errors between 0.0015 and 0.006 with 10 000 samples. Likelihood weighting is the better choice for these networks: it is as accurate as Gibbs sampling or more, and about 30 times faster, because each sample is a single pass in topological order, whereas every Gibbs sweep evaluates the Markov blanket of every unobserved variable. Gibbs sampling pays off when the evidence is very unlikely, so that likelihood weights degenerate, which does not happen with the moderately sized CPTs here.

method	samples	MAE, 2 evid.	MAE, 5 evid.	MAE, 10 evid.
Gibbs sampling	100	0.0444	0.0353	0.0159
Gibbs sampling	1000	0.0154	0.0124	0.0057
Gibbs sampling	10000	0.0061	0.0043	0.0025
Likelihood weighting	100	0.0467	0.0356	0.0183
Likelihood weighting	1000	0.0160	0.0131	0.0057
Likelihood weighting	10000	0.0049	0.0030	0.0015
Rejection sampling	100	0.1636	0.3041	0.3868
Rejection sampling	1000	0.0639	0.2514	0.3868
Rejection sampling	10000	0.0178	0.1673	0.3788

Table 4.16: Mean absolute error of approximate $P(\text{default} \mid \boldsymbol{e})$ against variable elimination (60 random test applicants per evidence size).

samples	2 evidence vars	5 evidence vars	10 evidence vars
100	0.04900	0.00100	0.00000
1000	0.04502	0.00077	0.00000
10000	0.04685	0.00073	0.00000

Table 4.17: Acceptance rate of rejection sampling.

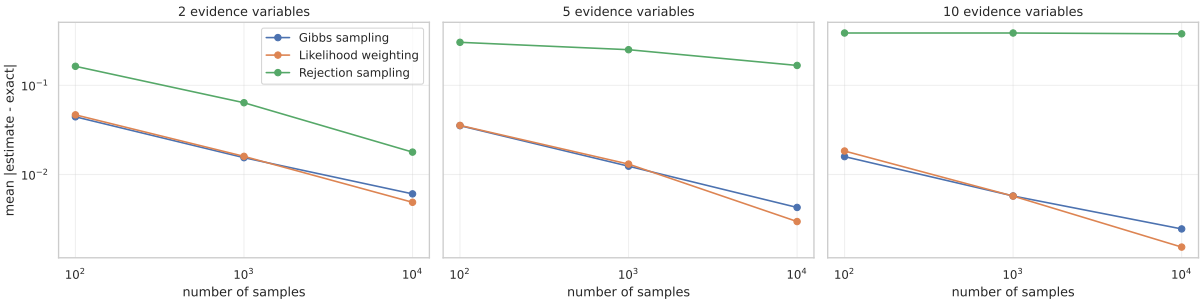


Figure 4.10: Mean absolute error of approximate inference against variable elimination (log-log scale).

4.7 Overall comparison

Table 4.18 and fig. 4.11 rank all fourteen models of the three families on the test set.

family	model	pr_auc	roc_auc	f2	recall	precision	accuracy
Traditional	Hist. Gradient Boosting	**0.9045**	**0.9459**	**0.8132**	**0.8512**	0.6901	0.8839
Traditional	Random Forest	0.8934	0.9361	0.8021	0.8068	0.7841	0.9092
Traditional	Decision Tree	0.8623	0.9180	0.7670	0.7856	0.7006	0.8797
Deep learning	MLP (entity embeddings)	0.8619 ± 0.0017	0.9200 ± 0.0017	0.7621 ± 0.0017	0.7956 ± 0.0088	0.6535 ± 0.0283	0.8626 ± 0.0105
Deep learning	TabNet	0.8612 ± 0.0026	0.9180 ± 0.0014	0.7568 ± 0.0039	0.7984 ± 0.0170	0.6288 ± 0.0398	0.8517 ± 0.0148
Traditional	SVM (RBF)	0.8561	0.9147	0.7703	0.8117	0.6398	0.8589
Traditional	k-NN	0.8163	0.8925	0.7501	0.7906	0.6228	0.8495
Bayesian network	BN - TAN	0.7895	0.8900	0.7390	0.8110	0.5453	0.8108
Bayesian network	BN - Expert DAG	0.7822	0.8712	0.7152	0.7736	0.5493	0.8117
Traditional	Logistic Regression	0.7512	0.8797	0.7360	0.7955	0.5665	0.8222
Traditional	Linear SVM	0.7443	0.8774	0.7339	0.7962	0.5589	0.8180
Bayesian network	BN - Hill-climb BIC	0.7332	0.8579	0.7033	0.7941	0.4826	0.7688
Bayesian network	BN - Naive Bayes	0.7061	0.8635	0.7149	0.7962	0.5076	0.7866
Traditional	Gaussian Naive Bayes	0.6244	0.8503	0.7140	0.8096	0.4850	0.7704

Table 4.18: All model families on the test set, ranked by PR-AUC (F2, recall, precision and accuracy at the tuned threshold); deep models are mean ± sd over 5 seeds.

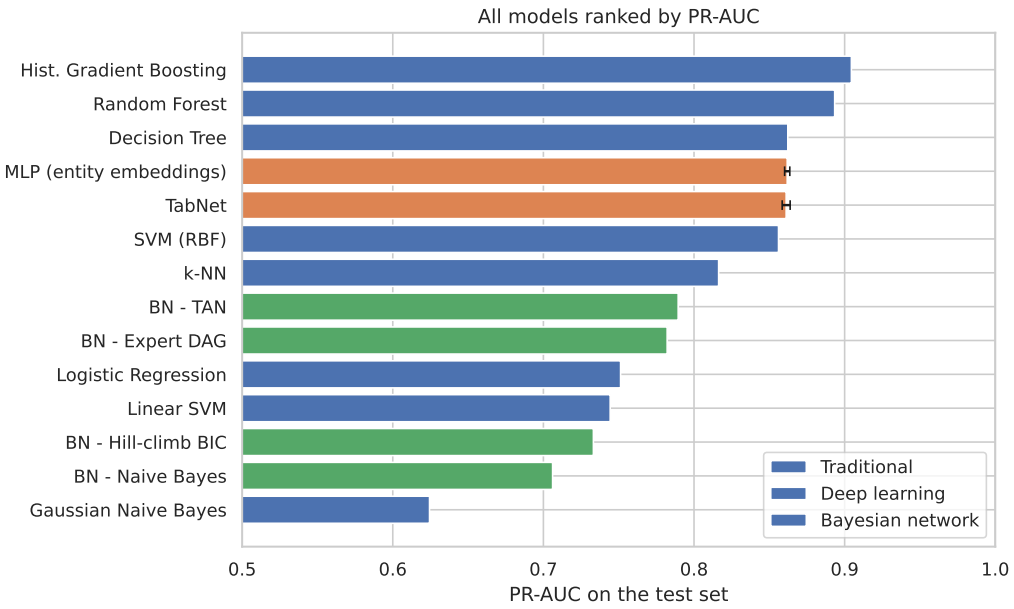


Figure 4.11: All models ranked by test PR-AUC; the error bars of the deep models are the standard deviation over five seeds.

5 Discussion

5.1 Comparison of the three approaches

Table 5.1 summarizes the strengths and weaknesses observed for the three families.

Approach	Strengths	Limitations
Traditional pipeline	Best accuracy (gradient boosting); fast to train (seconds); every preprocessing decision is explicit and testable; mature tooling for tuning and thresholding	Linear and distance-based models depend heavily on hand-made preprocessing; ensembles are harder to explain than a single model
Deep learning	Learns representations of categorical variables end to end; beats every linear model without manual interaction terms; the learned embeddings are reusable features	Below the tree ensembles on this medium-sized table; many training hyper-parameters; slower (TabNet especially); less transparent
Bayesian network	Explicit, inspectable assumptions; exact answers to any conditional query, in both predictive and diagnostic directions; native handling of missing features	Requires discretization, which loses information; CPT size grows exponentially with the number of parents; weakest classifier of the three families

Table 5.1: Strengths and limitations of the three model families on this problem.

5.2 What mattered most

Three decisions had a large effect, and most others had almost none.

1. **The model family.** Moving from a linear model to a tree ensemble raised the test PR-AUC from 0.75 to 0.90, more than every preprocessing choice combined.
2. **Scaling, for distance- and margin-based models.** It decides whether k-NN and the SVM are usable at all.
3. **The decision threshold.** For a fixed model, moving the threshold trades recall against precision far more than any re-weighting or resampling strategy does. Tuning it for F2 is a direct, transparent way to encode the higher cost of a missed defaulter.

In contrast, the imputation method, the treatment of the seven impossible values, the scaler variant and the resampling method changed PR-AUC by less than the fold-to-fold noise. Reporting these negative results matters: they show which parts of the pipeline deserve attention in practice.

5.3 Choice of metric

Accuracy would have been misleading for this problem. For example, the random forest has a higher test accuracy than gradient boosting at their tuned thresholds, but a lower recall; a lender who cares about missed defaulters should prefer the latter. PR-AUC ranks models by how well they separate defaulters across all thresholds, and F2 makes the cost asymmetry explicit when choosing one threshold. The weight $\beta = 2$ is itself a business assumption: a lender with a different ratio between the cost of a default and the lost margin of a rejected applicant should choose a different β, or minimize the expected cost directly.

5.4 Limitations and threats to validity

- **One dataset and one split.** All conclusions come from one public dataset with a single stratified test split. Cross-validation on the training set agrees with the test results, but the dataset's provenance (country, period, lender) is undocumented, so the absolute numbers may not transfer to other portfolios.
- **No temporal information.** Credit models are normally validated on later loans than they are trained on. The dataset has no dates, so a random split may slightly overstate performance under drift.
- **Limited search spaces.** The grids are deliberately small to keep *Run all* within the limits of a free Colab session. The headroom experiment (table 4.10) suggests this does not change the conclusions for the best family, but other families might gain somewhat from wider searches.
- **Fixed choices.** The IQR factor $k = 1.5$, the four engineered features, the MLP training schedule and the Bayesian-network parent limit were set by convention rather than tuned.
- **Fairness.** The features include age, and home ownership acts as a proxy for wealth. A deployed credit model must be audited for disparate impact, which is outside the scope of this assignment.
- **Calibration.** Class weights and threshold tuning shift the predicted probabilities, so the scores should be calibrated before they are read as default probabilities, for example to price a loan.

6 Conclusion and future work

This project built and compared three complete approaches to predicting loan default on the Credit Risk Dataset.

The **traditional pipeline** is fully configurable, from imputation to the classifier, and every option was evaluated with controlled cross-validated ablations. Scaling decides the fate of distance- and margin-based models, engineered features, IQR clipping and a one-hot grade help the linear models, and PCA and univariate feature selection never help reliably on this low-dimensional table. After tuning, histogram-based gradient boosting is the best model, with a test PR-AUC of 0.9045. At its F2-tuned threshold it detects 85.1% of the defaulters with a precision of 69.0%. A wider hyper-parameter search and a stacked ensemble did not improve on it, which indicates that the result is close to what these features allow.

The **deep learning pipeline** with entity embeddings outperforms every linear model and produces reusable learned features, but it stays below the tree ensembles, in line with the literature on tabular data.

The **Bayesian network**, implemented from scratch for the talented-engineering extension, is the least accurate classifier but the most transparent model. TAN is the best structure for prediction, whereas a better model of the joint distribution, as learned by hill climbing on BIC, does not translate into better classification; comparing exact query answers with empirical frequencies exposes the independence assumptions of each structure. The inference experiments reproduce the textbook behaviour: variable elimination is exact and fast on these small networks, likelihood weighting and Gibbs sampling converge, and rejection sampling collapses once many variables are observed.

Future work.

- Calibrate the probabilities (isotonic or Platt scaling) and choose the threshold by minimizing an explicit expected loss.
- Explain individual decisions with SHAP values, as lenders must justify rejections.
- Audit the model for disparate impact across age groups.
- Learn Bayesian-network parameters with expectation maximization over the incomplete rows, and try continuous (conditional Gaussian) nodes to avoid discretization.
- Validate on a time-ordered split if dated loan data become available.

References

[1] Sercan Ö. Arik and Tomas Pfister. TabNet: Attentive interpretable tabular learning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 35, pages 6679–6687, 2021.

[2] Leo Breiman. Random forests. *Machine Learning*, 45:5–32, 2001.

[3] Nitesh V. Chawla, Kevin W. Bowyer, Lawrence O. Hall, and W. Philip Kegelmeyer. SMOTE: Synthetic minority over-sampling technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002.

[4] C. K. Chow and C. N. Liu. Approximating discrete probability distributions with dependence trees. *IEEE Transactions on Information Theory*, 14(3):462–467, 1968.

[5] Corinna Cortes and Vladimir Vapnik. Support-vector networks. *Machine Learning*, 20:273–297, 1995.

[6] Jerome H. Friedman. Greedy function approximation: A gradient boosting machine. *The Annals of Statistics*, 29(5):1189–1232, 2001.

[7] Nir Friedman, Dan Geiger, and Moises Goldszmidt. Bayesian network classifiers. *Machine Learning*, 29:131–163, 1997.

[8] Robert Fung and Kuo-Chu Chang. Weighing and integrating evidence for stochastic simulation in Bayesian networks. In *Uncertainty in Artificial Intelligence 5*, pages 209–219. North-Holland, 1990.

[9] Stuart Geman and Donald Geman. Stochastic relaxation, Gibbs distributions, and the Bayesian restoration of images. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 6(6):721–741, 1984.

[10] Léo Grinsztajn, Edouard Oyallon, and Gaël Varoquaux. Why do tree-based models still outperform deep learning on typical tabular data? In *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, volume 35, 2022.

[11] Cheng Guo and Felix Berkhahn. Entity embeddings of categorical variables, 2016.

[12] Sergey Ioffe and Christian Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 448–456, 2015.

[13] Ian T. Jolliffe and Jorge Cadima. Principal component analysis: A review and recent developments. *Philosophical Transactions of the Royal Society A*, 374(2065):20150202, 2016.

[14] Guolin Ke, Qi Meng, Thomas Finley, Taifeng Wang, Wei Chen, Weidong Ma, Qiwei Ye, and Tie-Yan Liu. LightGBM: A highly efficient gradient boosting decision tree. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[15] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, 2009.

[16] laotse (Kaggle user). Credit risk dataset. Kaggle, 2020. https://www.kaggle.com/datasets/laotse/credit-risk-dataset.

[17] Guillaume Lemaître, Fernando Nogueira, and Christos K. Aridas. Imbalanced-learn: A Python toolbox to tackle the curse of imbalanced datasets in machine learning. *Journal of Machine Learning Research*, 18(17):1–5, 2017.

[18] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. In *International Conference on Learning Representations*, 2019.

[19] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, Zeming Lin, Natalia Gimelshein, Luca Antiga, et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems*, volume 32, 2019.

[20] Fabian Pedregosa, Gaël Varoquaux, Alexandre Gramfort, Vincent Michel, Bertrand Thirion, Olivier Grisel, Mathieu Blondel, Peter Prettenhofer, Ron Weiss, Vincent Dubourg, Jake Vanderplas, Alexandre Passos, David Cournapeau, Matthieu Brucher, Matthieu Perrot, and Édouard Duchesnay. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[21] Donald B. Rubin. *Multiple Imputation for Nonresponse in Surveys*. Wiley, 1987.

[22] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach*. Pearson, 4 edition, 2020.

[23] Takaya Saito and Marc Rehmsmeier. The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets. *PLoS ONE*, 10(3):e0118432, 2015.

[24] Gideon Schwarz. Estimating the dimension of a model. *The Annals of Statistics*, 6(2):461–464, 1978.

[25] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting. *Journal of Machine Learning Research*, 15:1929–1958, 2014.

[26] Olga Troyanskaya, Michael Cantor, Gavin Sherlock, Pat Brown, Trevor Hastie, Robert Tibshirani, David Botstein, and Russ B. Altman. Missing value estimation methods for DNA microarrays. *Bioinformatics*, 17(6):520–525, 2001.

[27] Stef van Buuren and Karin Groothuis-Oudshoorn. mice: Multivariate imputation by chained equations in R. *Journal of Statistical Software*, 45(3):1–67, 2011.

[28] C. J. van Rijsbergen. *Information Retrieval*. Butterworths, 2 edition, 1979.

[29] Nevin L. Zhang and David Poole. A simple approach to Bayesian network computations. In *Proceedings of the Tenth Canadian Conference on Artificial Intelligence*, pages 171–178, 1994.

A Project structure and usage

```
notebooks/credit_risk_pipeline.ipynb   Colab front-end (Run all)
modules/creditrisk/
  config.py        paths, dataset URL, column roles, random seed
  data.py          download, rule-based cleaning, stratified split
  eda.py           EDA tables and figures
  transformers.py  grade-wise imputer, IQR clipper, feature engineering
  pipeline.py      PipelineConfig -> scikit-learn / imbalanced-learn Pipeline
  experiments.py   ablation suites, grid search, threshold tuning, test evaluation
  evaluation.py    metrics, cross-validation, ROC / PR / confusion plots
  deep.py          entity-embedding MLP (PyTorch) and TabNet wrapper
  reporting.py     .npy / .h5 feature export, LaTeX tables
  bayesnet/        graph, factor, network, structure, inference, discretize
features/          saved feature sets (.npy) and credit_features.h5
reports/           report.pdf, LaTeX source, figures, generated tables
results/           CSV result of every experiment
tests/             pytest unit tests
```

Listing 2: Repository layout.

Running. On Google Colab, open the notebook link from section 1 and choose *Runtime* → *Run all*. Locally, install `requirements.txt` and PyTorch, then execute the notebook with Jupyter. The report is compiled with `latexmk-pdf-shell-escapereport.tex` in `reports/latex`.

B Unit tests

The package ships 21 `pytest` tests. They run in a few seconds and cover the following properties.

- The DAG rejects cycles, and its topological order and Markov blankets are correct.
- Variable elimination equals brute-force enumeration on random multi-parent networks, for queries with and without evidence (tolerance 10^{-12}).
- Rejection sampling, likelihood weighting and Gibbs sampling converge to the exact posterior (tolerance 0.01).
- Parameter learning recovers the generating CPTs from 200 000 samples.
- A Naive Bayes network gives the same posteriors as scikit-learn's `CategoricalNB`.
- Missing evidence is marginalized correctly.



- TAN and hill climbing produce valid DAGs, and the BIC trace is monotone.
- Cleaning, the grade-wise imputer and the IQR clipper behave as specified.
- Six representative pipeline configurations fit and predict, and invalid configurations are rejected.

C Saved feature files

Table C.1 lists the feature sets written by the notebook. Every set is stored as separate `.npy` arrays and also inside the single file `features/credit_features.h5`, with one HDF5 group per set.

Folder / HDF5 group	**Content**
`traditional_logreg`	Output of the tuned logistic-regression preprocessing (imputed, engineered, clipped, one-hot, standardized), with feature names in `meta.json`
`traditional_<best>`	Output of the preprocessing of the best traditional model
`traditional_pca95`	PCA projection retaining 95% of the variance, with explained-variance ratios
`deep_mlp_embeddings`	64-dimensional penultimate-layer representation of the reference MLP

Table C.1: Extracted feature sets (`X_train`, `X_test`, `y_train`, `y_test` in each).